

## 1부

---

# 딥러닝 기초 with 파이토치

### 1부에서 다루는 내용

---

- 머신러닝 모델링
- 딥러닝 기초
- 회귀 모델링
- 이진 분류 모델링
- 다중 분류 모델링
- 딥러닝 모델 성능 다루기
- 클래스 선언 방식

1~7일 차까지는 딥러닝을 학습하는 데 꼭 필요한  
기초 개념을 다룹니다. 머신러닝에서 시작해  
회귀, 분류, 모델링 절차, 모델 성능 관리,  
클래스 선언 방식 등 딥러닝의 눈으로 세상을 보는  
기반을 제공하는 내용들입니다. 특히 파이토치를  
활용해 직접 모델을 설계하고 학습시키는 흐름을  
반복함으로써 ‘코드가 어떻게 흘러가는지’  
자연스럽게 익힐 수 있도록 구성했습니다.

## 1일 차

# 머신러닝 모델링 빠르게 정리하기

### 이번 차시에서 주목할 키워드

- CRISP-DM
- 머신러닝
- 모집단과 표본
- 모델링 코드 절차

### 1일 차 학습 목표

- 머신러닝이 무엇이며 왜 필요한지, 그리고 모델, 모델링과 같은 기본 개념을 숙지합니다.
- 머신러닝을 체계적으로 정리해 주는 CRISP-DM 프로세스 여섯 단계를 설명할 수 있습니다.
- 실제 코드 실습을 통해 데이터 전처리 → 모델 설계 → 학습 → 예측 → 평가로 이어지는 모델링 절차를 익히고, 머신러닝 모델이 만들어지는 과정을 단계별로 이해합니다.

딥러닝을 올바르게 이해하고 활용하려면 그 바탕이 되는 머신러닝의 개념과 흐름을 먼저 익혀야 합니다. 1일 차에서는 본격적인 파이토치 딥러닝 학습에 앞서 머신러닝의 기본 개념과 실제 프로젝트 수행 절차를 살펴봅니다. 이 과정은 딥러닝 학습에 필요한 준비 운동의 일환이라고 생각하면 좋겠습니다.

머신러닝이 처음인 분은 여기서 소개하는 기본 개념과 실습을 꼼꼼하게 읽고 따라 해보기를 권합니다. 물론 머신러닝을 하나의 차시로 모두 익힐 수는 없지만, 머신러닝의 흐름과 구조 그리고 코드 작성 방식에 익숙해져야 딥러닝도 쉽게 이해할 수 있습니다. 이제부터 머신러닝과 딥러닝의 큰 흐름을 단계별로 익혀 보겠습니다.

## 머신러닝이란?

머신러닝/딥러닝 학습에 필요한 기초적인 몇 가지 용어를 알아봅니다. 많이 알려진 용어지만 여전히 정확히 사용되고 있지는 않습니다.

### 인공지능? 머신러닝? 딥러닝?

자주 혼동하는 세 용어를 정리합니다. 바로 인공지능(Artificial Intelligence, AI), 머신러닝(Machine Learning, ML), 딥러닝(Deep Learning, DL)입니다. 이 세 용어는 밀접하게 연결되어 있기 때문에 실무 현장에서는 혼용해 사용하는 경우가 많습니다. 하지만 학습을 시작하는 이 시점에서는 각각의 개념을 정확히 이해하고 넘어가는 것이 좋습니다.

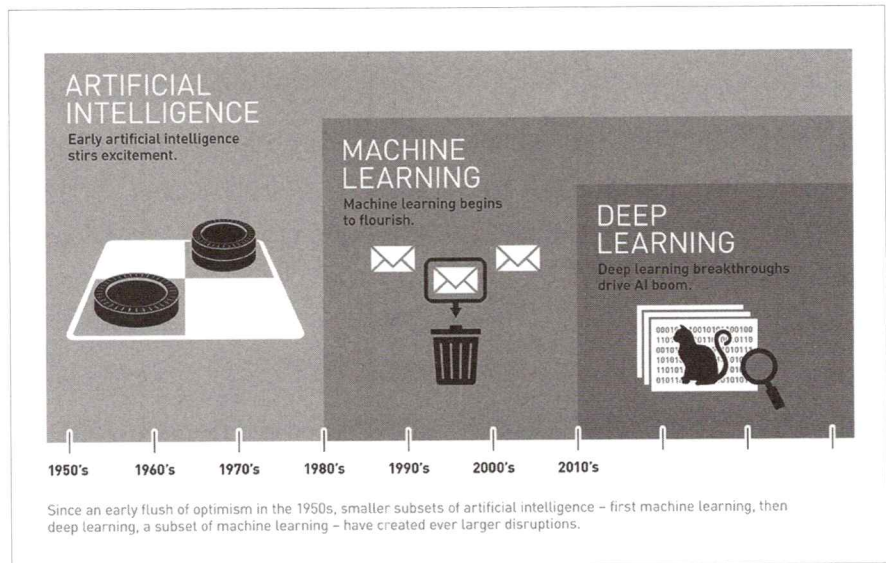
인공지능은 범위가 가장 넓은 개념으로 “인간처럼 사고하고 행동할 수 있는 기계를 만드는 것”을 목표로 하는 기술입니다. 초창기 인공지능은 규칙 기반(Rule-Based) 시스템이 중심이었습니다. 규칙 기반 시스템은 사람이 명시적으로 “이럴 땐 이렇게 하라”라고 컴퓨터에게 일일이 지시하는 방식입니다. 일반적으로 소프트웨어를 개발할 때 그리는 순서도(Flow Chart)가 규칙 기반 시스템의 대표적인 사

레입니다. 규칙이 많고 복잡할수록 사람의 개입은 많아지고 유지보수도 어려워집니다.

규칙 기반 시스템의 한계를 극복하는 과정에서 등장한 기술이 바로 머신러닝입니다. 머신러닝은 사람의 명시적인 지시 없이 데이터를 기반으로 컴퓨터가 스스로 패턴을 학습하고 의사결정을 내리는 기술입니다. 즉, 사람이 규칙을 하나하나 만들어 주지 않아도 머신 스스로 데이터를 기반으로 규칙을 추론합니다. 머신러닝은 인공지능의 한 분야로 분류할 수 있습니다.

딥러닝은 머신러닝의 하위 분야로 인공 신경망(Artificial Neural Network)을 기반으로 동작하는 기술입니다. 딥러닝 모델에서 입력 데이터는 여러 층(Layer)을 거치면서 점차 핵심적이고 추상적인 특징으로 변환됩니다. 복잡한 데이터를 변환하는 과정에서 딥러닝은 중요한 패턴을 찾습니다. 딥러닝은 이제 이미지에서 사물을 인식하고 음성 데이터를 분석해 발화 내용을 파악하고 텍스트에서 감정을 추론하거나 요약문을 생성하는 등 여러 작업에서 사용되고 있습니다.

인공지능, 머신러닝, 딥러닝은 [그림 1-1]이 보여 주듯이 서로 포함 관계에 있다 보니 세 용어가 종종 혼용되어 사용됩니다. 학문적, 실무적으로 정확한 구분이 필요할 때도 있지만, 일반적으로 사용할 때는 이 경계가 명확하지 않습니다. 특히 대학 강의에서 과목명은 머신러닝이지만, 배우는 내용은 주로 딥러닝(인공 신경망)인



[그림 1-1] 인공지능, 머신러닝, 딥러닝 비교(출처: <https://blogs.nvidia.com/blog/whats-difference-artificial-intelligence-machine-learning-deep-learning-ai/>)

경우가 많습니다. 딥러닝이 현재 머신러닝 분야에서 가장 활발히 연구되고 있고 응용 가능성도 높은 기술이기 때문입니다. 그래서 학계에서는 머신러닝이라고 쓰고 딥러닝이라고 부른답니다.

한편 실무에서는 종종 딥러닝과 머신러닝을 구분해 사용합니다. 딥러닝은 주로 인공 신경망 기반의 모델을 학습시키는 방법으로 정의하고 대규모 데이터를 처리하거나 비정형 데이터(예: 이미지, 음성, 텍스트)를 다룰 때 자주 활용합니다. 반면 머신러닝은 딥러닝을 제외한 나머지 알고리즘을 지칭하는 경우가 많습니다. 여기에는 전통적인 통계 모델들이 있습니다. 대표적으로 선형회귀(Linear Regression), 최근접 이웃(K-Nearest Neighbors), 의사결정나무(Decision Tree), 랜덤 포레스트(Random Forest), 그리고 XGBoost 같은 알고리즘들이 전통적인 통계 모델들입니다.

딥러닝이 머신러닝보다 무조건 뛰어난 방법은 아니라는 점이 중요합니다. 문제의 종류, 데이터의 양, 해석의 용이성, 컴퓨팅 자원 등에 따라 전통적인 머신러닝 기법이 더 적합한 경우도 많습니다. 따라서 이 두 가지 접근 방식을 ‘상호 보완적인 도구’로 이해하고 상황에 맞게 적절히 활용해야 합니다.

## 머신러닝의 종류

머신러닝은 모델이 데이터를 기반으로 스스로 패턴을 학습하고 이를 바탕으로 예측하거나 결정을 내리는 기술입니다. 그런데 모델이 데이터를 학습하는 방식은 모두 똑같지 않습니다.

학습 방식과 문제의 성격에 따라 머신러닝은 지도학습(Supervised Learning), 비지도학습(Unsupervised Learning), 강화학습(Reinforcement Learning), 세 가지로 구분합니다. 세 방식 모두 데이터에 대한 접근 방법이 다르고 활용 분야도 다양합니다.

### 지도학습

먼저 머신러닝의 대표적인 학습 방식인 지도학습(Supervised Learning)입니다. ‘지도’라는 단어가 암시하듯이 이 방식은 마치 선생님이 학생에게 정답을 알려 주며 학습을 이끄는 행위와 비슷합니다. 데이터를 이용해 모델을 학습시킬 때 입력 데이터와 함께 정답에 해당하는 목표값(target)도 제공합니다.

이메일 스팸 필터링 시스템을 만든다고 해봅시다. 이때 시스템의 입력값은 이메일의 내용이고 정답(목표값)은 ‘스팸이다’ 또는 ‘스팸이 아니다’와 같은 값입니다. 모델은 정답이 달린 수많은 이메일 데이터를 보면서 어떤 표현이나 구조가 스팸 메일에서 자주 등장하는지 학습합니다. 학습을 마친 모델은 새로운 이메일이 들어오면 이제 학습한 패턴에 따라 그것이 스팸인지 아닌지 예측할 수 있습니다.

지도학습은 정답 유형에 따라 다시 둘로 나뉩니다.

- 회귀(Regression): 정답이 숫자(연속형 값)일 때. 예: 집값 예측, 온도 예측 등
- 분류(Classification): 정답이 범주(클래스)일 때. 예: 합격/불합격, 고양이/강아지 분류 등

### 비지도학습

비지도학습(Unsupervised Learning)은 어떻게 다를까요? 지도학습이 정답(목표값)을 주면서 학습시키는 방법이라면 비지도학습은 이름 그대로 목표값이 주어지지 않은 상황에서 모델 스스로 데이터를 관찰하고 그 안에서 패턴을 찾는 학습 방법입니다. 사람이 “이게 정답이야”라고 알려 주지 않기 때문에 모델은 스스로 입력 데이터의 유사성을 기준으로 그룹을 나누거나 데이터의 숨은 구조를 파악합니다.

예를 들어 수많은 고객 데이터를 분석하면서 “이 고객과 저 고객이 비슷한 구매 행동을 보인다”라고 그룹화할 수 있다면 마케팅으로 각각의 그룹에 맞는 맞춤 전략을 세울 수 있습니다. 이처럼 비슷한 특성에 따라 데이터를 묶는 작업을 클러스터링(Clustering, 군집화)이라고 합니다.

또한 비지도학습은 차원 축소(Dimensionality Reduction) 작업에도 활용됩니다. 예를 들어 비지도학습은 수백 개의 특징이 있는 데이터를 2, 3차원으로 축소해 시각화하거나 노이즈(Noise)를 제거해 더 의미 있는 정보를 추출할 수 있습니다.

비지도학습은 우리가 아직 잘 모르는 데이터의 내부 구조나 숨은 패턴을 발견할 때 매우 유용한 도구입니다.



#### 입력값과 목표값(정답)의 다양한 이름들

모델을 학습시킬 때 데이터는 입력값과 목표값으로 표현됩니다. 그런데 이 단어와 유사한 용어가 아주 많습니다. 일부만 적어 보면 다음과 같습니다.

- 입력값: input, x, feature(특징, 특성), 요인, 독립변수
- 목표값: output, y, target, label, 결과, 종속변수, ground truth, 정답

이 용어들은 통계학, 머신러닝, 데이터 마이닝 등 여러 분야에서 사용합니다. 이 책에서는 주로 입력값( $x$ , feature)과 목표값( $y$ , target)으로 표기합니다. 그리고 입력값과 목표값을 변수(입력 변수, 목표변수)로 표기하기도 합니다.

## 강화학습

마지막으로 소개할 방식은 강화학습(Reinforcement Learning)입니다. 이 방식은 지도학습이나 비지도학습과는 접근 방식이 다릅니다. 강화학습에서는 모델이 정적인 데이터만 바라보는 것이 아니라, '환경'과 직접 상호작용하면서 학습합니다.

강화학습은 마치 게임과 비슷합니다. 모델이 어떤 행동을 취하면 환경은 그에 대한 보상(또는 벌점)을 제공합니다. 모델은 이 보상을 최대화하기 위해 최적의 행동 전략(Policy)을 스스로 찾아 학습합니다. 행동심리학에서 말하는 '보상 심리'와 매우 유사한 동작입니다. 한마디로 보상으로 바람직한 행동을 강화하는 것이죠.

쉬운 예로 체스나 바둑과 같은 게임을 생각해 봅시다. 머신러닝 모델은 게임을 여러 번 진행하면서 이기고 지는 경험을 합니다. 이때 모델이 두는 어떤 수에 일정한 보상이나 벌점을 주면 점차 나은 수를 선택할 가능성이 높아집니다. 2016년 이 세돌과의 바둑 대결로 잘 알려진 알파고(AlphaGo)는 실제로 강화학습으로 훈련했던 대표적인 인공지능입니다. 현재 강화학습은 게임 외에도 로봇 제어, 자율주행, 자동화된 금융 트레이딩 등 다양한 분야에서 응용되고 있습니다.

[표 1-1]은 지도, 비지도, 강화학습을 비교한 표입니다.

**[표 1-1]** 지도, 비지도, 강화학습 비교

| 학습 방식 | 데이터 제공 방식      | 예                   |
|-------|----------------|---------------------|
| 지도학습  | 입력+목표값 제공      | 예측, 분류, 스팸 필터링      |
| 비지도학습 | 입력만 제공, 목표값 없음 | 클러스터링, 차원 축소, 이상 탐지 |
| 강화학습  | 환경과 상호작용+보상    | 게임, 로봇 제어, 자율주행     |

이 책은 지도학습을 중심으로 딥러닝 모델을 학습합니다. 즉, 입력값(feature)과 목표값(target)이 주어진 데이터 세트(Dataset)를 기반으로 딥러닝 모델이 어떻게 목표값을 맞는지 단계적으로 살펴봅니다.

다만 마지막 차시에서는 비지도학습의 대표 모델인 오토인코더(Autoencoder)를 사용해 목표값 없이도 데이터를 표현하고 재구성하는 방법을 알아봅니다.

머신러닝의 다양한 학습 방식 가운데 “어떤 방법이 더 뛰어나다”라고 하기보다는 문제의 특성과 목적에 따라 적절한 방식을 선택하는 일이 더 중요합니다. 이번 차시에서 배울 머신러닝의 기본 개념을 잘 이해한다면 향후 딥러닝 학습이 훨씬 더 수월해질 겁니다.

## 모델과 모델링

### 모델

머신러닝에서 말하는 모델(Model)은 데이터에서 발견한 패턴을 수학적으로 표현한 구조(수학식)입니다. 모델은 단순히 데이터를 저장하거나 요약하는 것이 아니라, 데이터 속에 숨겨진 규칙성과 의미 있는 관계를 찾아내어 앞으로 발생할 새로운 데이터를 예측하거나 추론할 수 있도록 만든 도구입니다.

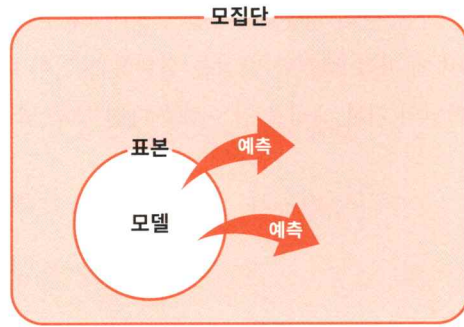
예를 들어 매장의 과거 판매 데이터를 이용해 다음 주의 판매량을 예측하거나 고객 데이터를 분석해 특정 고객이 어떤 제품을 구매할 확률이 높은지 알아내는 등의 작업이 모두 모델을 통해 이루어집니다. 다시 말해 모델은 데이터를 분석해 얻은 지식이나 경험을 수학적 구조로 담아낸 결과물이라고 할 수 있습니다.

### 모델링

모델을 만드는 과정이 바로 모델링(Modeling)입니다. 모델링할 때 가장 중요한 목표는 오차(Error, Loss)를 최소화하는 일입니다. 오차란 예측값과 목표값의 차이를 의미하며 모델이 얼마나 정확한지 평가하는 기준입니다. 이 오차를 줄이기 위해서는 데이터를 잘 이해하고 그에 적합한 알고리즘을 선택하며 모델의 구조나 하이퍼파라미터(Hyperparameter)를 잘 조정해야 합니다. 참고로 하이퍼파라미터는 모델을 튜닝(조정)할 때 사용하는 요소입니다.

모델링은 단순히 기술적인 작업을 넘어 데이터와 알고리즘, 그리고 문제 자체의 이해를 요구하는 종합적인 작업입니다. 단지 계산을 잘하는 것으로 좋은 모델을 만들 수는 없습니다. 어떤 문제를 해결하려는 것인지, 그 문제를 풀기 위해 어떤 데이터를 활용할 수 있는지, 어떤 방식으로 해석하고 적용할 수 있는지를 함께 고려해야 합니다.

모델의 가장 중요한 목적은 표본을 기반으로 전체(모집단)를 잘 추정하는 일입니다. 그 전에 ‘표본’과 ‘모집단’의 의미가 무엇인지 정확히 이해하는 일이 필요합니다.



[그림 1-2] 모집단과 표본

- 표본(Sample): 현재 가지고 있는 데이터, 즉 과거에 수집된 데이터의 일부를 말합니다. 표본은 전체 데이터 집합인 모집단에서 추출한 일부 데이터입니다. 모델은 이 표본 데이터를 이용해 패턴을 학습합니다.
- 모집단(Population): 실제로 예측하려는 모든 데이터 집합입니다. 전체 데이터에는 과거와 현재뿐 아니라, 아직 관측되지 않은 미래의 데이터도 포함됩니다. 왜냐하면 모델을 사용하는 궁극적인 목적은 미래에 발생할 사건이나 값을 예측하는 데 있기 때문입니다.

모델과 모델링의 개념을 이해했으니 다음은 머신러닝 프로젝트의 전체 흐름을 체계적으로 관리할 수 있는 방법론을 알아보겠습니다.

## 머신러닝 프로젝트의 수행 절차

머신러닝 프로젝트는 단순히 데이터를 불러와 모델을 학습시키는 것에 국한하지 않습니다. 머신러닝 프로젝트는 어떤 문제를 해결할지 명확히 정의하고 데이터를 이해하면서 전처리하고 적절한 모델을 선택하고 평가하고 실제로 활용할 수 있도록 배포하는 일련의 과정입니다. 그리고 머신러닝 프로젝트의 전체적인 흐름을 체계적으로 안내하는 방법론이 바로 CRISP-DM입니다.

CRISP-DM은 Cross Industry Standard Process for Data Mining의 약자로, 1990년대 말 유럽연합(EU)의 지원을 받아 IBM과 여러 데이터 마이닝 전문 기업들이 협력해 개발한 표준 프로세스입니다. 1990년대라니 꽤 오래되었죠? 하지만 지금도 기업과 기관에서 널리 사용하는 방법론입니다. 실제로 필자가 강의나 컨설팅을 다니며 만난 기업들의 머신러닝 프로젝트 역시 이 CRISP-DM을 기본 틀로 삼고 있었습니다.

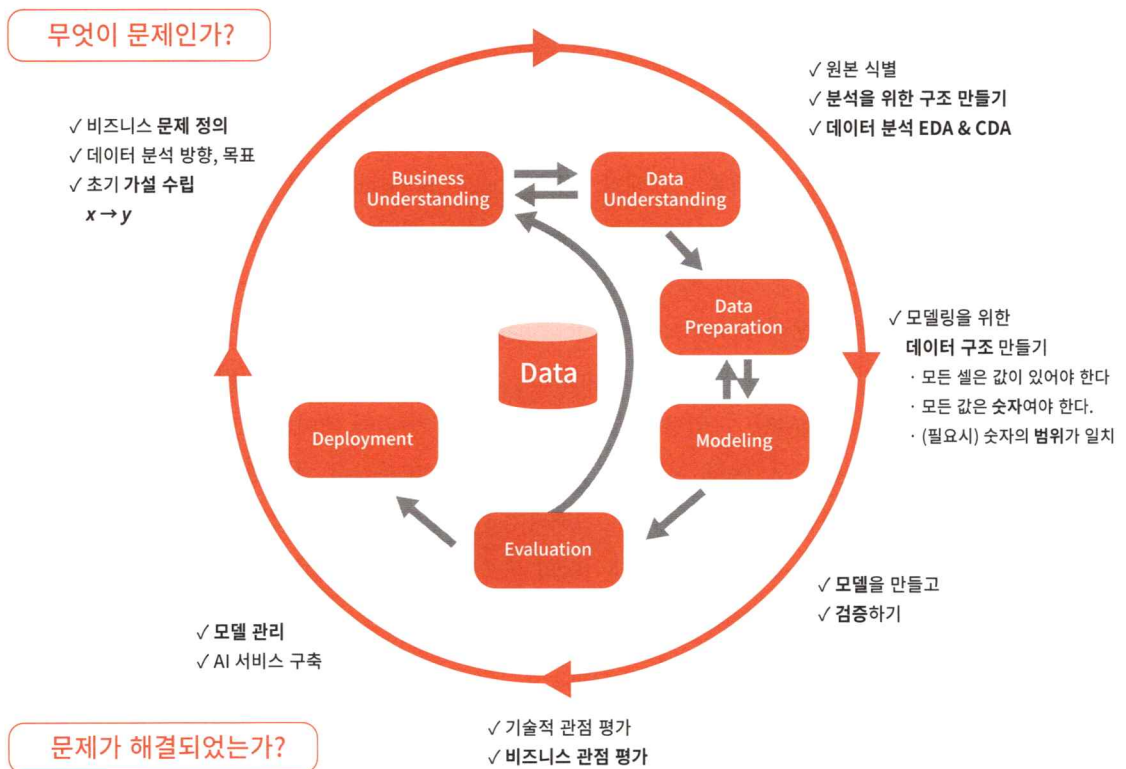
CRISP-DM은 현업에서 신뢰하는 실무 프레임워크입니다. 데이터 분석 및 인공지능 프로젝트를 처음부터 끝까지 끌고 가는 데 있어 훌륭한 나침반 역할을 합니다.

## CRISP-DM 살펴보기

CRISP-DM은 ①비즈니스 이해(Business Understanding), ②데이터 이해(Data Understanding), ③데이터 준비(Data Preparation), ④모델링(Modeling), ⑤평가(Evaluation), ⑥배포(Deployment) 프로세스로 구성되어 있습니다.

[그림 1-3]은 CRISP-DM 프로세스를 한눈에 볼 수 있도록 정리한 다이어그램입니다.

각각의 프로세스를 하나하나 살펴보겠습니다.



[그림 1-3] CRISP-DM 방법론

## 1. 비즈니스 이해

첫 번째 단계는 무엇보다 문제를 명확히 정의하는 일입니다(Business Understanding). “우리는 무엇을 해결하려고 하는가?”라는 질문을 중심으로 비즈니스 문제를 데이터 분석 문제로 변환하는 작업을 합니다. 예를 들어 매출을 늘리고 싶다는 목표가 있다면 이는 고객의 구매 패턴을 분석하고 최적의 마케팅 전략을 찾는 문제로 구체화할 수 있습니다. 이 단계에서는 방향성과 목표를 정립하고 초기에 해결할 가설을 수립하는 일이 핵심입니다.

가설 구조는 보통  $x \rightarrow y$ ( $x$ 이면  $y$ 이다)와 같은 논리 문장(전제와 결론)의 형태입니다. 여기서  $x$ 는 요인을,  $y$ 는 결과를 뜻합니다. 어떤 요인으로 인해 결과가 발생한다는 구조입니다. 예를 들면 “고객의 나이( $x$ )에 따라서 구매액( $y$ )이 달라진다”라고 가설을 구성할 수 있습니다(또는 “고객의 나이( $x$ )에 따라서 구매액( $y$ )에 차이가 있다”라고 쓰기도 합니다).

## 2. 데이터 이해

문제를 정의했다면 이제 데이터를 이해해야 합니다(Data Understanding). 어떤 데이터가 있는지, 데이터는 얼마나 신뢰할 수 있는지, 그리고 데이터의 주요 특성은 무엇인지 분석합니다. 이를 위해 탐색적 데이터 분석(Exploratory Data Analysis, EDA)과 확증적 데이터 분석(Confirmatory Data Analysis, CDA) 같은 기법을 써서 데이터의 분포와 가설의 타당성 여부를 파악합니다. 이 프로세스는 “이 데이터를 가지고 원하는 문제를 해결할 수 있을까?”라는 질문에 답하기 위해 데이터를 파악하는 중요한 단계입니다. 모델링을 본격적으로 수행하다 보면 사람은 대체로 모델의 구조, 기술에 집중하는 경향이 있습니다. 그러다가 데이터의 비즈니스적 의미를 잊습니다. 따라서 데이터 이해 단계에서 데이터의 분포와 통계량을 살펴보고 비즈니스의 의미를 파악하고 정리하는 일은 매우 중요합니다.

## 3. 데이터 준비

모델링을 위해서는 데이터가 특별한 구조를 갖도록 준비해야 합니다(Data Preparation). 데이터 준비는 데이터를 모델링에 적합한 형태로 가공하는 단계입니다. 여기서는 데이터를 정리하고 결측값을 처리하며 범주형 데이터를 숫자형 데이터로 변환하는 작업을 합니다. 또한 필요에 따라서는 숫자형 데이터의 범위를 맞추는 작업(Scaling)을 수행하기도 합니다. 데이터 준비는 모델 성능에 영향을 미치므로 꼭

끔한 처리가 필요합니다.

#### 4. 모델링

이제 모델을 설계하고 학습시킬 차례입니다(Modeling). 문제에 따라 회귀, 분류, 클러스터링 등 적절한 알고리즘을 선택하고 학습 데이터를 기반으로 패턴을 학습 시킵니다. 여러 모델을 실험하면서 성능을 비교하고 과적합 여부나 예측 정확도, 처리 시간 등 다양한 요소를 고려해 최적의 모델을 선택합니다. 이 단계는 우리가 다루는 인공지능 기술의 ‘핵심’이며 가장 흥미로운 부분이기도 합니다. 다만 좋은 모델을 만들기 위해서는 이전 단계에서 데이터 준비가 제대로 되어 있어야 합니다.

#### 5. 평가

모델이 잘 학습되었는지, 정말로 문제를 해결할 수 있는지 점검하는 단계입니다(Evaluation). 단순히 정확도가 높다고 해서 좋은 모델이라고 할 수는 없습니다. 처음 정의한 “비즈니스 문제 해결에 모델이 실질적으로 도움이 되는가?”라는 관점에서도 평가가 이뤄져야 합니다. 기술적 지표뿐만 아니라 현업에서 실제로 사용할 수 있는지, 성능이 지나치게 특정 집단에 편향되지는 않았는지 등도 함께 고려해야 합니다.

#### 6. 배포

모델이 평가 단계를 통과했다면 이제 실전에 투입합니다(Deployment). 배포 단계에서는 모델을 운영 시스템에 적용하거나 사용자에게 결과를 보여 주는 애플리케이션에 연결하는 등의 작업이 이뤄집니다. 배포 후에도 성능을 지속적으로 모니터링해야 합니다. 시간이 지나면서 데이터의 특성이 바뀌거나 외부 환경이 변하면 기존 모델의 성능도 떨어질 수 있기 때문입니다. 이런 현상을 모델 드리프트(Model Drift)라고 하며 이를 감지하고 대응하는 일도 실무에서는 매우 중요한 작업입니다.

CRISP-DM은 각각의 단계가 유기적으로 연결된 프로세스를 통해 데이터 분석 프로젝트를 효율적으로 수행하도록 도와줍니다. 데이터 분석이나 모델링 과제를 수행할 때 어디서부터 시작할지 막막하다면 이 CRISP-DM의 프로세스대로 수행해 보길 권장합니다.

## 두 가지 중요한 질문

CRISP-DM은 데이터 분석과 인공지능 모델링을 효과적으로 수행하기 위한 프로세스이자 방법론입니다. 이 방법론에서 가장 중요한 핵심은 ‘비즈니스 문제 해결’입니다. 그래서 필자는 강의할 때마다 CRISP-DM을 별도의 이름으로 부르곤 합니다. 즉, ‘데이터와 AI를 활용한 비즈니스 문제 해결 방법론’이라고 말이죠. 좀 길지만, CRISP-DM을 이처럼 명확히 설명하는 명칭은 없습니다.

이어서 CRISP-DM을 적용해 과제를 수행할 때 비즈니스 문제 해결 관점에서 꼭 고려해야 할 두 가지 중요한 질문에 대해 설명하겠습니다.

### 첫 번째 질문: “무엇이 문제인가?”

데이터 분석과 모델링 기술의 궁극적인 목적은 비즈니스 문제를 해결하는 데 있습니다. 따라서 해결해야 할 문제를 명확히 정의하는 일이 가장 우선되어야 합니다. 문제가 무엇인지도 모른 채 복잡한 기술이나 알고리즘을 고민하는 일은 시간과 자원의 낭비가 될 수 있습니다. 먼저 비즈니스 목표가 무엇인지 정확히 이해해야 합니다. 그래야 이 목표를 데이터 분석 과제로 자연스럽게 연결할 수 있습니다.

### 두 번째 질문: “문제가 해결되었는가?”

아무리 멋진 분석 보고서나 복잡한 인공지능 모델을 만들었다 해도 최초로 정의했던 비즈니스 문제를 해결하지 못한다면 그 결과는 무의미합니다. CRISP-DM의 평가 단계에서는 반드시 이 질문에 답할 수 있어야 합니다. 배포 이후에도 인공지능 시스템이 운영 환경에서 비즈니스 문제를 해결하는지 지속적으로 모니터링해야 합니다. 이는 성공적인 모델링과 분석이 실질적인 가치를 창출하기 위해서 반드시 거쳐야 할 과정입니다.

이 두 질문은 단순히 CRISP-DM의 흐름을 따라가는 데서 멈추지 않고 모든 데이터 분석 프로젝트의 본질에 집중하자는 메시지를 담고 있습니다. 복잡한 기술보다 ‘무엇을 왜 분석하는가’를 기억하는 것이 더 중요합니다. 이것이 바로 CRISP-DM 방법론에서 가장 중요한 초점입니다.

독자에게 당부하고 싶은 이야기가 있습니다. 차시마다 새로운 알고리즘을 배우고 실습을 구현하기 전에 다시 이 페이지로 돌아와 CRISP-DM의 프로세스를 살펴

보기 바랍니다. 머신러닝/딥러닝의 모든 알고리즘과 기법은 결국 비즈니스 문제를 해결하기 위해 필요합니다. CRISP-DM 프로세스를 다시 살펴보는 일은 알고리즘의 세부 내용에 자칫 빠질 수도 있는 독자로 하여금 전체적인 프로세스 관점에서 모델링을 바라보게 하고 비즈니스 문제 해결이라는 근본적인 목적에 초점을 맞추는 데 많은 도움이 될 것입니다.

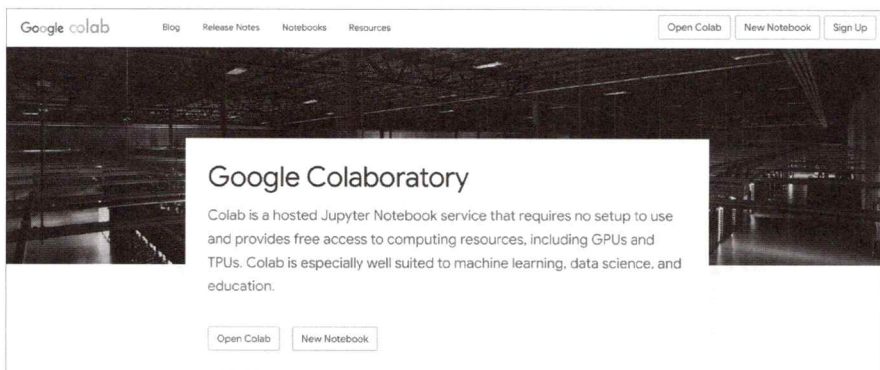
## 개발 환경 코랩 소개

딥러닝 실습을 위해서는 먼저 코드를 작성하고 실행할 수 있는 개발 환경이 필요합니다. 그런데 파이썬 설치부터 라이브러리 세팅, GPU 환경 구성까지 처음부터 모든 것을 손수 준비하려면 생각보다 많은 시간이 걸립니다. 이때 사용할 수 있는 가장 간편하고 강력한 도구가 바로 구글 코랩(Colab, Colaboratory)입니다.

### 구글 코랩이란?

코랩은 구글에서 제공하는 클라우드 기반의 무료 개발 환경으로 별도의 설치 없이 웹 브라우저만 있다면 바로 파이썬 코드를 실행할 수 있습니다. 무엇보다 GPU와 TPU 자원까지 무료로 제공하기 때문에 머신러닝이나 딥러닝을 학습하는 입장에서 매우 매력적인 도구입니다.

코랩은 주피터 노트북(Jupyter Notebook) 환경과 동일한 인터페이스를 제공하기 때문에 이 편집기를 사용해 본 사람이라면 금방 적응할 수 있습니다. 물론 처음 사용하는 사람이라도 간단한 사용법만 익히면 어렵지 않게 사용할 수 있습니다. 이 책의 실습 코드는 모두 코랩 환경에서 작성합니다.



[그림 1-4] 구글 코랩(<https://colab.research.google.com>)

구글 코랩은 다음과 같은 특징이 있습니다.

- 클라우드 기반 개발 환경: 코랩은 로컬 컴퓨터에 파이썬이나 라이브러리를 따로 설치할 필요가 없습니다. 구글 클라우드 서버에서 코드를 실행하기 때문에 인터넷만 연결되어 있다면 어디서든 바로 실습할 수 있습니다. 개인 PC의 성능이 떨어져도 코랩에서는 전혀 문제가 되지 않습니다.
- 무료로 제공하는 GPU 및 TPU: 코랩은 고사양 컴퓨팅 자원이 필요한 딥러닝 작업을 위해 GPU(Graphics Processing Unit)와 TPU(Tensor Processing Unit)를 지원합니다. 설정에서 런타임 유형만 변경하면 손쉽게 GPU를 사용할 수 있기 때문에 대용량 데이터를 훨씬 빠르게 처리할 수 있습니다.
- 주피터 노트북과 동일한 인터페이스: 코랩은 주피터 노트북과 거의 동일한 인터페이스를 제공하므로 사용자는 익숙한 환경에서 코드를 작성하고 결과를 확인할 수 있습니다. 코드 셀, 마크다운 셀(코랩에서는 '텍스트 셀'이라고 함)을 활용해 코드를 작성하면서 동시에 문서화할 수 있습니다.
- 구글 드라이브와의 통합: 코랩은 구글 드라이브(Google Drive)와 완벽히 연동됩니다. 작업한 파일을 드라이브에 저장하면 언제든지 다시 열어 볼 수 있고 파일 공유도 간편합니다. 또한 외부 데이터를 불러오거나 저장하는 일도 쉽게 처리할 수 있습니다.
- 다양한 라이브러리 내장: 코랩에서는 머신러닝과 데이터 분석에 필요한 주요 라이브러리(NumPy, Pandas, Matplotlib, SciPy, sklearn 등)들이 기본으로 설치되어 있습니다. 따라서 사용자는 별도의 설치 과정 없이도 바로 작업할 수 있습니다. 이 책에서 다루는 파이토치(Pytorch) 라이브러리도 이미 설치되어 있습니다.

코랩은 지금 딥러닝을 공부하는 여러분이 가장 쉽고 빠르게 시작할 수 있는 실습 환경입니다.

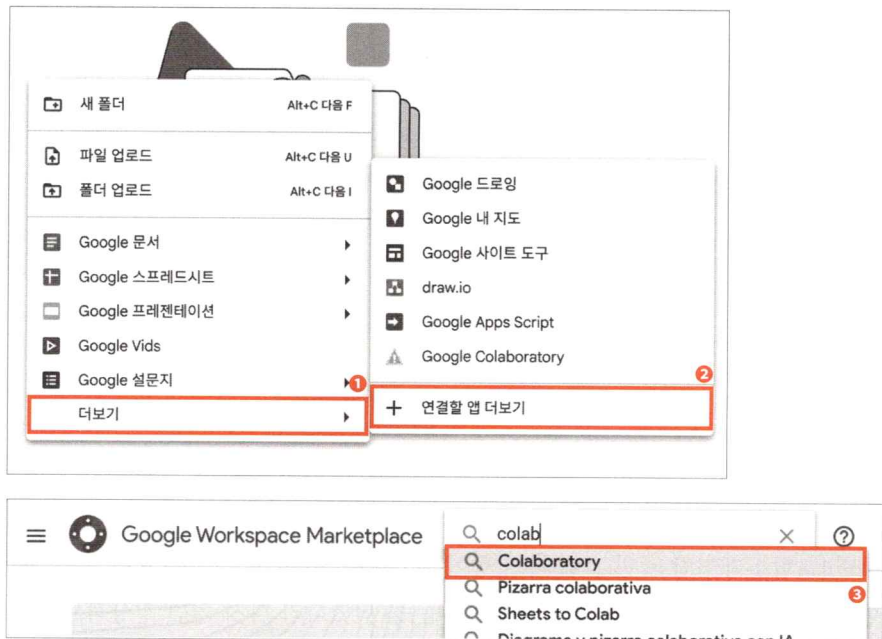
## 코랩 설치하기

코랩을 사용하려면 먼저 두 가지를 준비해야 합니다. 하나는 구글 ID입니다. 구글 ID는 구글 홈페이지에 접속해 [로그인] - [계정 만들기]를 진행하면 얻을 수 있습니다.

또 하나는 구글 드라이브에서 코랩을 설치해야 합니다. 코랩은 다음과 같은 방법으로 설치합니다. 먼저 구글 ID로 로그인하고 구글 드라이브로 이동합니다(<https://>

drive.google.com). 구글 드라이브에서 왼쪽 윗부분에 있는 <+ 신규> 버튼을 클릭하면 팝업 메뉴가 나옵니다. 팝업 메뉴에서 [새 폴더]를 클릭해 폴더를 하나 만듭니다 (새 폴더의 이름은 자유롭게 정합니다). 지금 생성한 폴더가 앞으로 코랩에서 작성할 코드 파일(\*.ipynb)을 저장할 공간입니다.

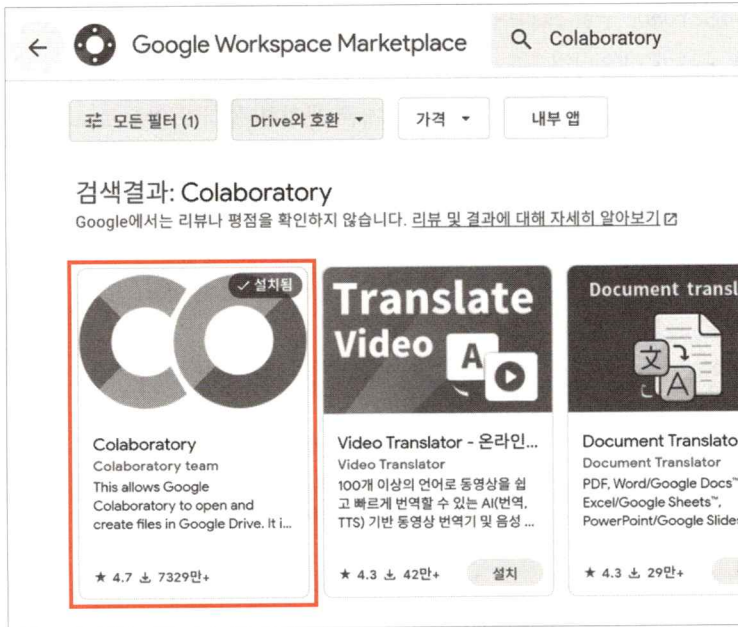
계속해서 생성한 폴더에서 [그림 1-5]의 순서에 따라 코랩을 설치합니다.



[그림 1-5] 구글 코랩 설치하기 1

- ❶ ❷ 구글 드라이브에 생성한 폴더에서 마우스 오른쪽 버튼을 클릭하고 [더보기] - [+ 연결할 앱 더보기] 메뉴를 선택합니다. 그럼 [Google Workspace Marketplace] 창이 열립니다.
- ❸ [Google Workspace Marketplace] 창의 검색 폼에서 'Colaboratory'를 입력하고  키를 누릅니다..

[그림 1-6]과 같은 화면이 나오면 'Colaboratory' 카드를 클릭해 설치합니다.



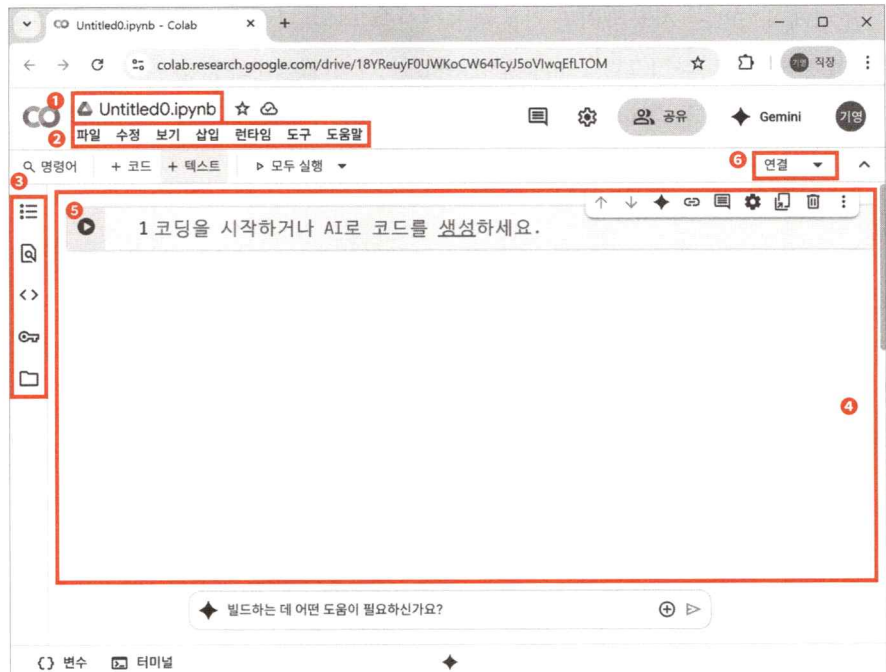
[그림 1-6] 구글 코랩 설치하기 2

설치는 금방 끝납니다. 다시 한번 생성한 폴더에서 마우스 오른쪽 버튼을 클릭하고 [더보기] 메뉴를 선택하면 [Google Colaboratory] 메뉴가 보입니다. 이 메뉴를 클릭하면 [그림 1-기과 같은 빈 코랩 파일 창이 열립니다. 코랩 파일 창이 보이지 않는다면 브라우저(구글 크롬이나 윈도우 Edge 등)를 닫았다가 다시 열면 정상적으로 동작합니다.

## 코랩의 기본 화면

[그림 1-기은 코랩의 기본 화면 구성입니다. 주피터 노트북과 유사한 화면 구성으로 처음 사용자도 금방 익숙해집니다

- ❶ 코드 파일의 이름을 입력합니다.
- ❷ 기본 메뉴 구성입니다.
- ❸ 왼쪽 패널: 목차, 검색, 파일 등을 간단히 확인할 수 있게 아이콘을 모아 놓은 곳입니다.
- ❹ 본문 편집 영역: 실제로 코드나 텍스트를 입력하고 실행하는 공간입니다. 코드 셀, 텍스트 셀을 작성하고 실행 결과를 바로 확인할 수 있어 실습과 문서 작성을 동시에 진행할 수 있습니다.



[그림 1-7] 구글 코랩의 기본 화면

- ⑤ 코드 셀: 코드와 주석을 입력하는 곳입니다. 앞으로 가장 많이 사용할 요소입니다.
- ⑥ 연결: <연결> 버튼을 누르면 이 파일을 구글에서 제공하는 클라우드와 연결합니다. <연결> 버튼을 누르지 않더라도 파일을 열고 코드를 실행하면 '연결'을 먼저 수행합니다. 만약 이미 연결되어 있다면 <연결> 버튼이 아니라 <RAM, 디스크> 버튼으로 바뀌고 현재 연결된 시스템 상태를 보여 줍니다.

코랩은 기본 기능 외에도 많은 기능이 있습니다. 추가 기능은 필요할 때마다 설명하겠습니다.

## GPU 연결하기

코랩의 장점 중 하나는 무료로 GPU를 이용할 수 있다는 점입니다. GPU는 비싼 하드웨어이므로 노트북이나 PC에 장착하기엔 부담이 됩니다. 하지만 딥러닝 모델링을 수행하는 데 GPU가 없다면 수행 시간이 오래 걸릴 수 있습니다. 구글 코랩에서는 GPU를 무료로 제공합니다. 물론 무한정 쓸 수는 없고 사용에 일정한 제약이 있습니다.

코랩에서 GPU를 사용하려면 기본 메뉴에서 [런타임] - [런타임 유형 변경]을 차례로 선택하면 됩니다. [그림 1-8]과 같이 GPU를 선택하는 대화상자가 나옵니다. 가장 일반적인 GPU는 ‘T4 GPU’이므로 이를 선택하고 <저장> 버튼을 클릭하면 GPU를 사용할 수 있습니다.



[그림 1-8] 런타임 유형 변경, GPU 선택

GPU가 무료라고 해서 마음껏 쓸 수 있는 장치는 아닙니다. 그래서 꼭 필요할 때에 한정해 사용합니다. GPU가 필요하지 않는데도 ‘GPU 런타임’을 선택하면 코랩은 “GPU를 사용하지 않으면 CPU 런타임으로 변경하세요”라는 팝업 경고문을 계속 내보냅니다. 이 책에서도 필요한 실습에 한정해 GPU 런타임을 사용합니다.

## 코랩 사용법 간단하게 알아보기

코랩의 코드 셀과 텍스트 셀(마크다운 셀)의 사용법은 주피터 노트북과 매우 유사합니다. 셀과 셀 경계에 마우스를 갖다 대면 <+ 코드>, <+ 텍스트>라는 버튼이 생기는데, 이를 클릭하면 새로운 셀을 간단하게 추가할 수 있습니다.

코드 셀에서는 파이썬 코드와 주석을 작성합니다. 여기서는 실행 방법만 간단히 확인하고 넘어갑니다.

다음과 같이 코드 셀에 다음과 같이 파이썬 코드를 입력하고 **Shift** + **Enter** 키를 누릅니다.

CODE

```
print('Hello world!')
```

OUTPUT

```
Hello world!
```

코드 셀 하단에 Hello world!라는 텍스트를 출력하고 커서는 다음 셀로 이동합니다.  
코드 셀을 실행하는 방법은 다음과 같은 방법이 있습니다.

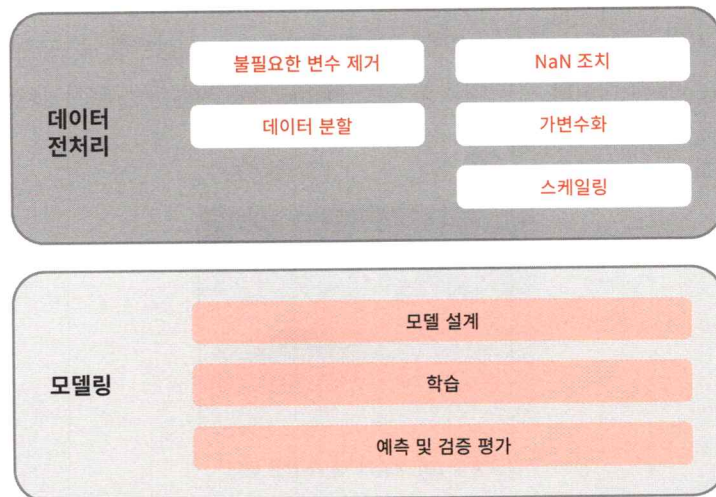
- **Ctrl + Enter** 혹은 코드 셀 왼쪽에 있는 ▶(플레이 버튼): 코드 셀을 실행합니다.
- **Shift + Enter**: 코드 셀을 실행하고 다음 셀로 이동합니다.
- **Alt + Enter**: 코드 셀을 실행하고 빈 코드 셀을 추가합니다.

다양한 실행 방법이 있으니 필요에 맞게 사용하기 바랍니다. 이 정도의 동작만 익혀도 당장 코랩에서 실습하는 데 지장이 없습니다. 좀 더 세부적인 내용은 필요할 때마다 설명합니다.

## 머신러닝 코드 작성 절차

실제 머신러닝 모델을 만들기 위해 코드를 작성할 차례입니다. 그런데 아무리 좋은 도구와 알고리즘이 있더라도 코드를 어떤 흐름으로 작성할지 미리 머릿속에 그려두지 않으면 쉽게 길을 잃을 수 있습니다. 따라서 모델링을 본격적으로 구현하기에 앞서 머신러닝 코드를 작성하는 전반적인 절차를 먼저 정리해 보겠습니다.

머신러닝 코드의 작성 절차는 [그림 1-9]와 같이 크게 두 부분으로 나눌 수 있습니다.



[그림 1-9] 머신러닝 코드 작성 절차

- 데이터 전처리: 모델을 제대로 학습할 수 있도록 데이터를 정리하고 준비하는 단계입니다. 데이터를 잘 준비하는 것이 모델의 성능을 높이는 데 굉장히 중요한 역할을 합니다. 어쩌면 모델링보다 더 중요한 단계라고 할 수 있습니다.
- 모델링: 준비된 데이터를 바탕으로 모델을 설계하고 학습시키며 예측과 평가까지 수행하는 단계입니다. 이때 다양한 머신러닝 알고리즘을 이용해 모델을 만듭니다.

이제 파트별로 어떤 작업을 포함하는지 좀 더 구체적으로 살펴보겠습니다.

## 데이터 전처리

데이터 전처리는 모델을 학습시킬 수 있는 형태로 데이터를 정리하는 과정입니다. 이 단계는 마치 요리를 하기 전 재료를 다듬는 일과 같습니다. 아무리 좋은 요리사라도 재료 손질이 제대로 되어 있지 않으면 좋은 요리를 기대하기 어렵듯이 데이터 전처리가 허술하면 모델 성능도 떨어질 수밖에 없습니다.

데이터 전처리에는 다음과 같은 3가지 핵심 요건이 있습니다.

- 모든 셀에는 값이 있어야 한다.
- 그 값은 모두 숫자여야 한다.
- 필요에 따라서는 각 열(Column)의 값(숫자) 범위를 맞춰 주어야 한다.

이 세 요건을 충족해야 비로소 모델링 코드를 안정적으로 실행할 수 있습니다. 여기서 ‘셀(Cell)’은 데이터 세트 각각의 요소(Element)를 지칭하는 용어입니다. 마치 엑셀의 셀과 같은 의미로 받아들이면 됩니다.

|   |   |   |
|---|---|---|
|   |   |   |
| 셀 | 셀 | 셀 |
|   |   |   |
|   |   |   |
|   |   |   |

[그림 1-10] 엑셀의 ‘셀’과 같은 의미

### 첫 번째 요건: “모든 셀에는 값이 있어야 한다”

모델링하기 전에 결측값(Missing Value)을 식별하고 이를 처리하는 작업을 의미합니다. 결측값이란 데이터에서 값이 비어 있거나 수집되지 않은 상태를 말합니다. 예를 들어 어떤 사람의 키나 나이를 조사했는데, 응답하지 않아 셀에 값이 누락되었다면 이를 결측값이라고 합니다. 결측값은 삭제하거나 특정 값(평균, 최빈값, 중앙값 등)으로 대체해 셀이 비어 있지 않도록 해야 합니다. 결측값은 NAN(Not a Number) 또는 NA(Not Available)라고도 표기합니다.

결측값을 조치하는 방법은 크게 두 가지입니다.

- 채우기: 결측값 자리에 다른 값을 채웁니다. 숫자형 feature에는 평균(Mean), 범주형 feature에는 최빈값(Mode) 등 다른 값으로 추정해 채우는 방법이 여러 가지 있습니다.
- 삭제하기: 결측값이 포함된 열을 삭제하거나 행을 삭제하는 방법입니다. 결측값 비율이 높고 열 자체가 중요한 정보가 아니라면 열을 삭제하는 것을 검토합니다. 그러나 비교적 결측값 비율이 적고 실제 운영에서도 결측값이 포함된 행을 삭제하고 처리해도 무방하다면 그때는 행 삭제를 검토합니다. 하지만 삭제를 결정할 때는 항상 주의해야 합니다. 먼저 채우는 방법을 검토하고 그래도 여의치 않다면 삭제합니다.

결측값을 조치하는 방법을 결정할 때는 통계적인 접근보다는 비즈니스 관점에서 이 방법을 적용하는 것이 타당한지 검토해야 합니다.

### 두 번째 요건: “값은 모두 숫자여야 한다”

데이터 세트의 모든 값은 숫자여야 모델링에서 사용할 수 있습니다. 보통 범주형 데이터의 경우 모델이 이해할 수 있는 숫자로 변환하는 과정이 필요합니다. 이를 위해 가변수(Dummy Variable)화 혹은 다른 말로 원-핫 인코딩(One-Hot Encoding)이라는 기법을 사용해 범주형 데이터를 숫자로 변환합니다. 여기서 “왜 꼭 숫자로 변환해야 하지?”라는 의문이 들 수 있습니다. 앞에서 모델은 데이터에 있는 패턴을 찾아 이 패턴을 수학적식으로 정리해 놓은 것이라고 설명했습니다. 수학적식에서 범주형 값은 계산할 수 없기 때문에 숫자로 변환하는 작업이 반드시 필요합니다.

### 세 번째 요건: “각 열(변수)의 값 범위를 맞춰야 한다”

데이터를 학습할 때는 입력값(feature)마다 범위(Scale)가 다르면 모델 학습에 영향을 미칠 수 있으므로 범위를 맞춰 주어야 합니다. 스케일링(Scaling) 기법을 활용해 모든 입력값이 동일한 범위(스케일)를 가지도록 정규화나 표준화를 수행합니다.

- 정규화(Normalization): 입력값의 범위를 0~1 사이로 맞추는 방법입니다. 예를 들어 고객의 나이가 21~87세의 범위에 있다면 21세를 0, 87세를 1로 변환합니다. 결국 이 둘 사이에 있는 나이는 모두 '0.##'으로 표현됩니다. 정규화를 적용하면 모든 입력값의 최솟값은 0, 최댓값은 1이 되므로 범위가 일치하게 됩니다.
- 표준화(Standardization): 입력값의 평균을 0, 표준편차를 1로 맞추는 스케일링 기법입니다. 표준화를 적용하면 모든 입력값의 평균과 표준편차가 일치하게 됩니다.

스케일링과 관련해 한가지 오해가 있습니다. 스케일링을 적용하면 원래의 분포 모양이 변한다고 생각합니다. 스케일링은 범위를 맞추는 방법입니다. 그러므로 분포 모양은 스케일링 전이나 후 모두 동일합니다.

### 데이터 분할

데이터 전처리 과정에는 데이터의 분할 작업도 포함됩니다. 데이터 분할은 모델의 학습과 평가에서 기본이 되는 중요한 단계입니다. 데이터 세트를 적절히 분할해 학습 데이터(train), 검증 데이터(validation), 그리고 테스트 데이터(test)로 나눕니다.

분할된 데이터를 이용해 모델의 일반화 성능을 평가할 수 있습니다. 일반화 성능은 학습할 때 사용하지 않은 새로운 데이터에서 모델이 얼마나 잘 작동하는지를 보여주는 지표입니다. 쉽게 말해 모델을 객관적으로 평가할 때의 성적이라고 할 수 있습니다. 어떤 모델이 학습 데이터에서 아무리 높은 정확도를 보여도 학습하지 않은 새로운 데이터에서 성능이 떨어진다면 그 모델은 실제 문제를 해결하기 어렵다고 평가를 받습니다.

학습, 검증, 평가 데이터 세트를 살펴보겠습니다.

- 학습 데이터: 모델을 학습시키기 위해 사용하는 데이터입니다. 이 데이터를 기반으로 모델은 데이터의 패턴과 규칙을 학습합니다. 모델링할 때는 충분한 학습 데이터를 확보하는 일이 중요합니다.

- 검증 데이터: 학습한 모델의 성능을 객관적으로 평가하기 위해서 검증 데이터 세트를 사용합니다. 학습할 때 사용하지 않은 검증 데이터로 예측하고 성능을 평가하기 때문에 객관적인 평가가 가능합니다.
- 테스트 데이터: 최종적으로 모델의 성능을 평가하기 위해 사용하는 데이터입니다. 학습이나 검증 과정에서 사용하지 않은 완전히 분리된 데이터로서 마치 실제 운영 과정에서 발생한 데이터인양 간주됩니다. 학습 과정에서 사용하지 않은 데이터이기에 보통 new data라고도 합니다.

데이터 분할은 모델이 특정 데이터에 치우치지 않고 다양한 상황에서 안정적으로 작동할 수 있도록 도와주는 과정입니다. 올바른 데이터 분할은 모델의 신뢰성과 성능 평가의 공정성을 높이는 데 중요한 역할을 합니다.

## 모델링

데이터 전처리를 완료했다면 이제 본격적인 모델링에 들어갑니다. 모델링은 말 그대로 모델을 만들고 학습시키고 성능을 평가하는 일련의 과정입니다.

### 모델 설계

모델링의 첫 번째 단계는 모델 설계입니다. 모델 설계는 문제를 해결하기 위해 적합한 알고리즘을 선택하고 이를 바탕으로 모델의 구조를 정의하는 작업입니다. 딥러닝은 구조를 자유롭게 설계할 수 있는 반면, 머신러닝은 정해진 구조 안에서 일부 설정만 조정해 모델을 구성합니다. 이들 옵션은 모델 학습과 성능에 영향을 미치는데, 이를 하이퍼파라미터(Hyperparameter)라고 합니다. 또한 하이퍼파라미터를 적절히 조정하며 성능을 향상시키는 과정을 하이퍼파라미터 튜닝(Hyperparameter Tuning)이라고 합니다.



#### 파라미터? 하이퍼파라미터?

파라미터와 하이퍼파라미터를 구분하는 것을 어려워하는 분들이 많습니다. 이 두 용어를 구분해서 이해하는 일은 중요합니다.

- 하이퍼파라미터(Hyperparameter): 학습을 하기 전에 사람이 직접 설정하는 값입니다. 예를 들어 의사결정나무(Decision Tree)로 모델을 설계할 때 트리의 크기를 결정하는 max\_depth 값을 지정하는데, 이 값이 바로 하이퍼파라미터입니다.

- 파라미터(Parameter): 모델이 학습을 통해 자동으로 찾아내는 값입니다. 예를 들어 의사결정나무 모델에서 각 노드의 분할 기준은 모델이 학습으로 찾은 파라미터입니다.

## 모델 학습

설계한 모델을 학습시키는 단계입니다. 학습 데이터에서 패턴을 찾아내고 이를 수학적 형태로 정리합니다. 학습 과정에서는 손실(Loss)을 최소화하기 위한 연산이 반복되는데, 데이터의 양이나 모델의 복잡도에 따라 많은 시간이 걸릴 수 있습니다.



### 오차(Error), 손실(Loss)?

머신러닝과 딥러닝에서 오차(Error)와 손실(Loss)은 거의 같은 의미로 사용합니다. 둘 다 예측값과 목표값 간의 차(Difference)를 의미하며 모델이 얼마나 잘못 예측했는지를 수치로 표현한 값입니다. 일반적으로 오차는 개별 데이터에 대한 차이, 손실은 이들을 종합한 전체적인 평가값입니다. 하지만 현장에서는 오차와 손실, 오차 함수와 손실 함수를 혼용해 사용합니다. 이 책에서는 오차를 계산하는 함수는 손실 함수, 개별 데이터의 차나 손실 함수로 계산된 값은 오차라고 하겠습니다.

## 예측 및 검증 평가

학습이 끝나면 모델이 잘 학습됐는지 검증하고 실제 문제 해결에 사용할 수 있을 만큼 성능이 나오는지 확인하는 단계로 넘어갑니다. 검증 데이터(Validation Dataset)로는 하이퍼파라미터를 조정하거나 해서 성능을 미리 점검하고, 테스트 데이터(Test Dataset)로는 모델을 운영 환경에서 사용할 때 얻을 것으로 기대하는 성능을 측정합니다.

이때 모델 종류에 따라 다양한 평가 지표를 사용합니다. 숫자를 예측하는 회귀 모델은 목표값과 예측값의 차(오차)를 평가하는 MSE(Mean Squared Error, 평균 제곱 오차), MAE(Mean Absolute Error, 평균 절댓값 오차), MAPE(Mean Absolute Percentage Error, 평균 절댓값 오차율) 등으로 평가합니다. 그리고 분류 모델 평가는 정분류율(Accuracy), 정밀도(Precision), 재현율(Recall) 등을 사용합니다.

다음 절에서는 이 절차를 바탕으로 실제 회귀와 분류 예제를 실습하면서 본격적으로 모델링 실력을 키워 보겠습니다. 머신러닝/딥러닝 입문자를 위해 이 책에서 처음 등장하는 코드는 비록 쉬운 부분일지라도 코드의 문법과 그 의미를 최대한 자세히 설명할 예정입니다.

## 회귀 모델링 예제: 광고비에 따른 매출 예측 모델

지금까지 머신러닝의 개념과 코드 작성 절차를 대략적이거나 익혔다면 이제는 실제로 모델링 예제를 따라하면서 감을 잡아 보는 시간이 필요합니다. 이번 절에서는 광고비 집행에 따라 매출액을 예측하는 회귀 모델을 만듭니다.



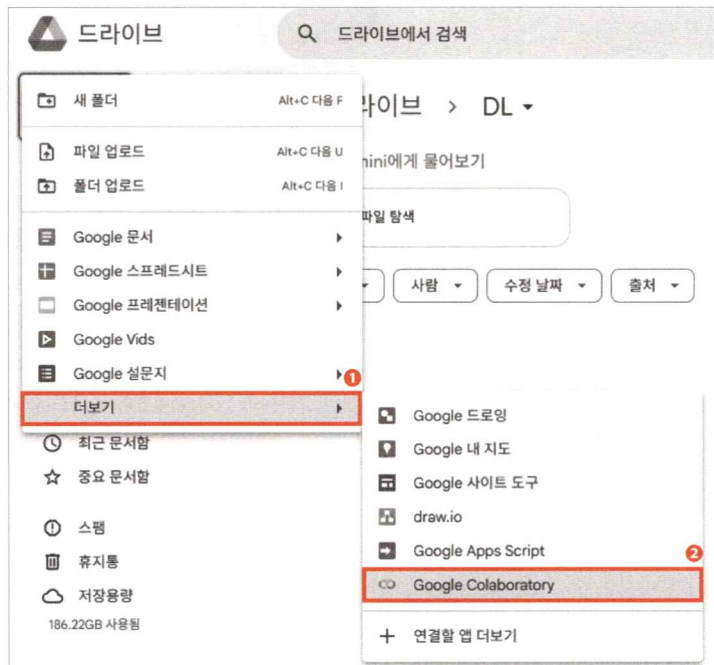
### 회귀 모델이란?

‘회귀(回歸, Regression)’라는 말은 원래 통계학에서 유래된 용어로 어떤 값이 평균으로 되돌아 간다는 뜻에서 시작되었습니다. 하지만 머신러닝에서 회귀는 ‘숫자를 예측한다’는 의미로 사용 됩니다.

회귀(Regression) 모델은 주어진 입력 데이터를 바탕으로 연속적인 수치값을 예측하는 데 사용하는 머신러닝 알고리즘입니다. 예를 들어 광고비에 따라 매출액을 예측하거나 기온에 따라 아이스크림 판매량을 예측하는 문제처럼 수치값을 추정할 때 회귀 모델을 사용합니다.

## 새 코랩 파일 생성하기

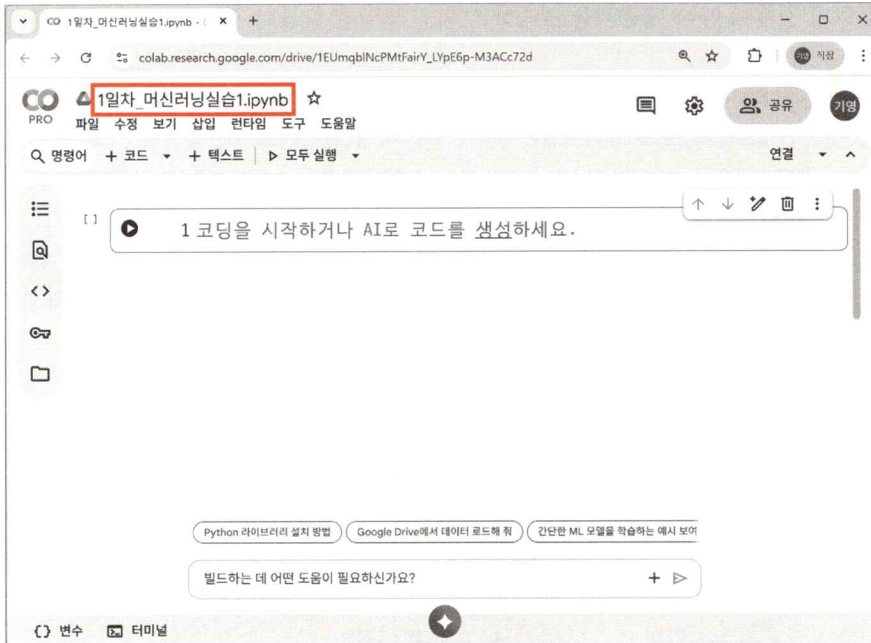
구글 드라이브로 이동합니다. 앞에서 만든 폴더에서 <+신규> 버튼을 클릭한 다음, [더보기] - [Google Colaboratory]를 선택합니다.



[그림 1-11] 신규 코랩 파일 생성

빈 코랩 파일 창이 나옵니다.

먼저 빈 코랩 파일 왼쪽 상단에 'Untitled0.ipynb'라고 되어 있는 파일 이름을 '1일차\_머신러닝실습1.ipynb'로 변경합니다.



[그림 1-12] 빈 코랩 파일



### 코랩 파일? 주피터 노트북 파일?

파이썬 코드 파일은 보통 확장자가 .py이거나 .ipynb 형식입니다. .py 파일은 일반적인 파이썬 코드 파일로서 텍스트 기반의 스크립트 파일입니다. .py 파일은 메모장이나 비주얼 스튜디오 코드 같은 에디터에서 열 수 있으며, 파일을 실행하면 파이썬 코드를 한 줄씩 순차적으로 수행합니다. 단순하고 가볍지만, 코드 설명이나 시각화 결과를 함께 보기엔 다소 불편할 수 있습니다. 반면 .ipynb 파일은 주피터 노트북 형식의 파일로서 코드는 물론 텍스트 설명(Markdown), 표, 그래프 등을 함께 기록할 수 있습니다. 특히 데이터 분석과 딥러닝 실습에서 코드 실행 결과를 바로 확인하고 코드에 설명을 곁들일 수 있어 매우 유용합니다. 구글 코랩도 .ipynb 형식을 기반으로 작동하는 웹 기반 주피터 노트북 환경입니다.

## 환경 준비

코드 파일의 처음 몇몇 셀에서는 주로 코드에서 사용할 라이브러리를 불러오거나 필요한 설정을 하거나 코드에서 반복해 사용할 함수를 정의합니다.

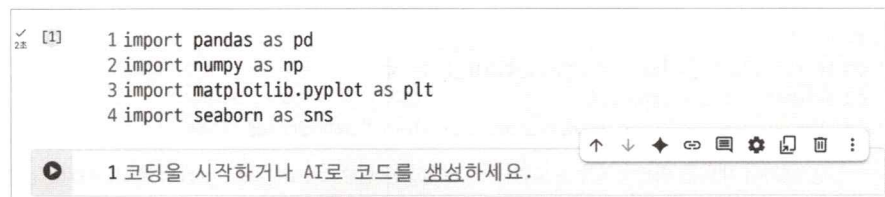
먼저 기본적인 라이브러리 4개를 불러옵니다.

### CODE

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

pandas, numpy는 데이터를 다루는 라이브러리이고, matplotlib과 seaborn은 데이터를 시각화할 때 사용하는 라이브러리입니다. 코드에서 사용할 때는 이 라이브러리들을 as 문법으로 pd, np, plt, sns 등의 별칭으로 사용합니다. 네 개의 라이브러리는 자주 사용하므로 항상 기본으로 불러온다고 생각하면 됩니다.

모두 작성했다면 셀에서 **Shift**+**Enter** 키를 누르거나 코드 셀 왼쪽에 있는 ▶ (플레이 버튼)을 클릭해 셀을 실행합니다. 정상적으로 불러왔다면 [그림 1-13]과 같이 셀 왼쪽에 실행 시간과 함께 ✓ 기호가 표시됩니다. 또한 실행 시간 옆에 보이는 숫자 [1]은 실행 순번으로서 코드 셀을 실행할 때마다 1씩 증가합니다.

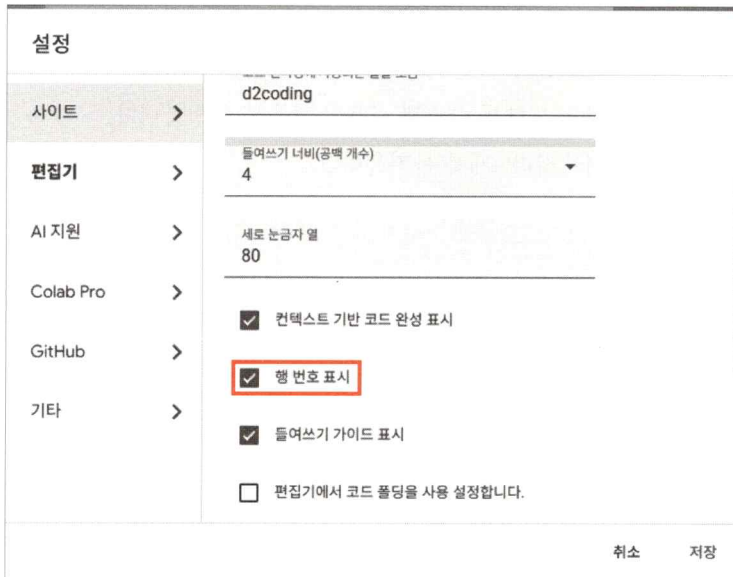


[그림 1-13] 코드 셀이 정상적으로 실행된 상태



### 셀 코드의 일련번호

셀 코드 앞에 1, 2, 3, 4라고 번호가 붙어 있는데, 이는 라인(행) 번호를 의미합니다. 이 라인 번호는 사용자가 언제든지 설정할 수 있습니다. 기본 메뉴에서 [도구] - [설정]을 누르면 [설정] 대화상자가 나옵니다. 대화상자 왼쪽에 있는 [편집기] 메뉴를 선택하고 설정 항목의 '행 번호 표시'를 체크하면 코드 셀에서 이 번호가 보입니다. 개발할 때 라인 번호를 확인하는 경우가 종종 있으므로 체크해 두기 바랍니다.



[그림 1-14] 코랩 설정에서 '행 번호 표시'

다음 셀에서 머신러닝 모델링에 필요한 sklearn(scikit-learn) 라이브러리 함수들도 불러옵니다. sklearn은 머신러닝을 위해 다양한 함수를 모아 놓은 유명한 라이브러리입니다.

학습이 끝난 모델의 성능평가  
(평가지표)

분류  
회귀

MSE -> RMSE  
MAE  
MAPE

$R^2 \rightarrow r^2\_score$

**CODE**

```

from sklearn.model_selection import train_test_split ①
from sklearn.metrics import * * (함부로 쓰는 거 아님) ②
from sklearn.preprocessing import StandardScaler, MinMaxScaler ③

```

- ① 데이터 세트를 학습과 테스트 데이터로 분할하는 함수 train\_test\_split을 sklearn 라이브러리에서 불러옵니다.
- ② 모델을 평가하는 sklearn 라이브러리의 지표 모듈(metrics)에서 모든 함수(\*)를 불러옵니다. 앞에서 '모델링'의 '예측 및 검증 평가'에서 소개한 MSE(mean\_squared\_error)나 MAE(mean\_absolute\_error) 같은 성능 평가 함수들이 지표 모듈 함수입니다.
- ③ sklearn의 전처리 모듈(preprocessing)에서 스케일링 함수(StandardScaler, MinMaxScaler)를 불러옵니다.

코드 셀을 실행해 필요한 라이브러리와 함수들을 불러왔습니다. 실제로 개발하다 보면 이외에도 더 많은 라이브러리와 함수를 불러와 사용합니다.

## 데이터 전처리

다음은 데이터를 모델링에 적합한 형태로 정리하는 전처리 단계입니다. 앞에서 전처리의 3가지 핵심 요건을 간단히 설명했습니다. 이를 떠올리면서 코드를 작성합니다.

### 데이터 준비

모델링에 사용할 데이터를 불러옵니다. 데이터 역시 코드 파일의 전반부에서 불러옵니다.

빈 코드 셀에서 다음과 같이 입력합니다.

#### CODE

```
path = 'https://bit.ly/advertising_csv' ①
data1 = pd.read_csv(path)              ②
data1.head()                           ③
```

#### OUTPUT

|   | TV    | Radio | Newspaper | Sales |
|---|-------|-------|-----------|-------|
| 0 | 230.1 | 37.8  | 69.2      | 22.1  |
| 1 | 44.5  | 39.3  | 45.1      | 10.4  |
| 2 | 17.2  | 45.9  | 69.3      | 9.3   |
| 3 | 151.5 | 41.3  | 58.5      | 18.5  |
| 4 | 180.8 | 10.8  | 58.4      | 12.9  |

- ① csv(Comma-Separated Values) 형식의 데이터를 불러올 수 있도록 경로를 지정합니다. 경로는 필자의 깃허브(Github)에 등록되어 있는 데이터의 링크 주소입니다. 이 주소를 변수 path에 저장합니다.
- ② 판다스의 read\_csv 메서드에서 파일 경로를 담은 변수를 지정하고 실행하면 데이터를 불러와 판다스의 데이터프레임 형식으로 저장합니다(data1).
- ③ 판다스의 데이터프레임 구조와 데이터의 일부를 조회할 때 head(n) 형식의 메서드를 사용하는데, 이때 n은 조회할 데이터의 건수를 의미합니다. head 메서드의 괄호에서 아무런 값도 지정하지 않으면 기본적으로 데이터 세트의 상위에 있는 5건의 데이터를 출력합니다.



#### 함수와 메서드

함수는 어떤 입력을 주면 그 입력을 계산해서 결과를 돌려주는 작은 코드 블록이라고 할 수 있습니다. 예를 들어 '더하기 함수'의 경우 두 수를 넣으면 두 수를 더한 결과를 돌려주는 역할을

합니다. 그래서 함수는 누구의 것도 아니고, 그냥 독립적으로 만들어 두고 필요할 때 불러 쓰는 도구입니다.

메서드는 겉으로 보면 함수와 비슷하게 동작합니다. 다만 차이가 있다면 메서드는 어떤 대상(객체라고 부름)에 붙어 그 대상을 다루는 일을 합니다. 예를 들어 df로 명명한 데이터프레임이 있을 때, 이 데이터의 일부를 보려면 df 뒤에 .head()를 붙이면 됩니다. 이때의 head가 바로 메서드입니다. 즉, 메서드는 ‘특정 대상을 위한 함수’라고 생각하면 이해하기 쉽습니다.

정리하면 함수는 독립적으로 동작하는 계산 규칙이고 메서드는 어떤 대상에 붙어 그 대상을 다루는 계산 규칙입니다.

광고 데이터를 불러왔으니 이제 데이터를 전처리하고 모델링해야 하지만, 그보다 데이터를 이해하는 일이 먼저입니다. 데이터를 잘 모른 채 모델링을 시작한다면 마치 눈을 감고 운전하는 것과 다를없습니다.

앞에서 코드 셀을 실행하면 [표 1-2]와 같은 형태의 결과가 나왔습니다.

[표 1-2] 광고비에 따른 매출 데이터

| TV    | Radio | Newspaper | Sales |     |
|-------|-------|-----------|-------|-----|
| 230.1 | 37.8  | 69.2      | 22.1  | 관측치 |
| 44.5  | 39.3  | 45.1      | 10.4  |     |
| 17.2  | 45.9  | 69.3      | 9.3   |     |
| 151.5 | 41.3  | 58.5      | 18.5  |     |
| 180.8 | 10.8  | 58.4      | 12.9  |     |

입력값(feature, x)                      목표값(target, y)

각각의 행은 분석 단위, 혹은 예측 단위입니다. 구체적으로 어떤 지역이나 어떤 기간의 데이터인지는 명시되지 않았지만, 행 하나를 단위 데이터로 보고 모델을 만들 수 있습니다.

Sales(매출액)는 예측하려는 목표값(target 또는 y)이고 TV, Radio, Newspaper는 광고비라는 입력값(feature 또는 x)들입니다. 이번에 생성할 회귀 모델의 목표는 세 광고비를 입력으로 받아 매출(Sales)을 예측하는 것입니다. 참고로 여기서 매출액의 단위는 1,000달러입니다.

## x, y 나누기

다음으로 전체 데이터에서 x와 y를 분리해야 합니다. 이 작업은 기술적으로 꼭 필

요한 것은 아니지만, 전처리나 모델링을 좀 더 깔끔하고 체계적으로 구성하기 위해 많이 사용합니다.

빈 코드 셀에서 다음 코드를 입력합니다.

CODE

```
target = 'Sales' ①  
x = data1.drop(target, axis=1) ②  
y = data1.loc[:, target] ③
```

- ① 변수 Sales를 target으로 지정합니다.
- ② drop 메서드는 데이터프레임에서 열 혹은 행을 삭제할 때 사용합니다. x=data1.drop(target, axis=1)은 target에 저장한 Sales 열을 data1에서 삭제하고 나머지 열은 x에 저장합니다. 이때 axis=1은 열을 의미합니다. 만약 행을 삭제하려면 axis=0으로 지정해야 합니다.
- ③ data1.loc[:, target]은 Sales 열만 조회하며 그 결과를 y에 저장합니다. 데이터프레임에 담긴 데이터를 조회할 때는 loc[행 조건, 열 조건] 형식으로 사용합니다. 행 조건 대신 :을 사용하면 모든 행을 조회하겠다는 의미입니다.

데이터를 이처럼 x, y로 나누면 이후 각각의 데이터 처리에 필요한 작업을 손쉽게 수행할 수 있습니다.

### 데이터 분할: train, validation

다음으로 데이터를 분할합니다. 이번 예제에서는 테스트 데이터는 이미 분리되어 있다고 가정하고 학습 데이터와 검증 데이터로만 나누겠습니다. train\_test\_split 함수는 sklearn에서 제공하는 데이터 분할 함수입니다. 앞에서 분할한 x와 y를 입력값으로 넣고 학습 데이터와 검증 데이터를 7:3의 비율로 지정합니다. 함수 이름이 train\_test\_split이어서 자칫 학습 데이터와 테스트 데이터를 분할한다고 생각할 수 있지만, 이 함수는 하나의 데이터 세트를 두 데이터 세트로 분할하는 함수라고 이해하는 게 좋습니다.

빈 셀에서 다음과 같이 입력합니다.

CODE

```
x_train, x_val, y_train, y_val = train_test_split(x, y, test_size = .3,  
                                                  random_state=20) ①
```

- ① test\_size=0.3은 전체 데이터의 30%를 검증 데이터로 사용한다는 의미입니다. random\_state=20은 결과의 재현성을 위해 설정합니다.

한 가지 주의할 점이 있습니다. 바로 함수가 반환하는 결과값을 저장할 변수의 위

치입니다. 함수의 반환값은 `x_train`, `x_val`, `y_train`, `y_val`, 총 4개의 변수에 저장됩니다. `x` 두 개(`x_train`, `x_val`)가 먼저 나오고, 다음에 `y` 두 개(`y_train`, `y_val`)가 나옵니다. 이 순서를 헷갈리는 사람이 많으니 유의하길 바랍니다.



#### 결과의 재현성이란?

재현성이란 동일한 코드를 여러 번 실행하더라도 항상 같은 결과가 나오도록 보장하는 특성을 말합니다. 머신러닝에서는 데이터 분할이나 모델 학습 과정이 무작위(Random)로 처리되는 경우가 많습니다. 이때 `random_state`(혹은 `random seed`)를 지정하면 결과의 일관성을 확보할 수 있습니다. `random_state`에 값을 지정하면 무작위로 처리되던 데이터 분할의 결과도 항상 일정하게 나옵니다.

`random_state` 설정은 실험 결과를 비교하거나 다른 사람과 결과를 공유할 때 매우 중요합니다. 필자도 강의할 때 `random_state` 코드를 사용합니다. 강사와 교육생이 동일한 코드를 실행했을 때 동일한 결과가 나오도록 하기 위함입니다.

## 스케일링

다음으로 전처리의 마지막 단계인 스케일링을 수행합니다. 스케일링은 거리 계산을 기반으로 패턴을 찾는 알고리즘에서는 필수적으로 사용합니다. 그러나 선형회귀나 의사결정나무 같은 알고리즘에서 스케일링 여부는 결과에 아무런 영향을 주지 않습니다.

빈 셀에서 다음과 같이 입력합니다.

#### CODE

```
scaler = MinMaxScaler() ①
x_train = scaler.fit_transform(x_train) ②
x_val = scaler.transform(x_val) ③
```

- ① 'MinMax' 방식의 스케일러를 선언합니다. `MinMaxScaler` 함수는 모든 값의 범위를 0~1 사이의 값으로 변환합니다.
- ② 학습 데이터(`x_train`)를 기준으로 최솟값과 최댓값을 찾고(`fit`), 0~1 사이의 값으로 변환(`transform`)합니다. 이를 한꺼번에 해주는 메서드가 `fit_transform`입니다.
- ③ 검증 데이터(`x_val`)는 학습 데이터의 최솟값과 최댓값을 기준으로 변환만 수행합니다. 이때는 `transform` 메서드를 사용합니다.

일반적으로 스케일링은 학습 데이터를 기준으로 설정하고(`fit`), 나머지 데이터 세트, 이를테면 검증 데이터나 테스트 데이터는 변환(`transform`)만 수행합니다. 다음

차시부터 시작할 딥러닝에서는 스케일링을 반드시 해야 합니다. 값의 크기가 학습의 속도나 안정성에 영향을 미치기 때문입니다. 따라서 여러분은 “스케일링은 기본적으로 전처리 작업이다”라고 이해하는 게 좋습니다.

여기까지가 머신러닝을 위한 데이터 준비 단계입니다. 코드를 직접 작성하는 것이 때로는 조금 귀찮은 작업처럼 여겨질 수도 있지만, 코드를 직접 손으로 하나씩 작성할 때 실력이 늡니다. 이어서 모델링 단계로 넘어갑니다.

## 모델링

### 학습 및 검증

이제 모델을 선언하고 학습시키고 검증 평가하는 단계입니다. 여기서 사용할 모델은 머신러닝에서 가장 기본이 되는 선형회귀(Linear Regression)입니다. 선형회귀는 직선 하나로 데이터를 설명하려는 아주 단순한 방식이지만, 그만큼 해석도 직관적이어서 딥러닝의 근간이 되는 알고리즘입니다.

#### CODE

```
from sklearn.linear_model import LinearRegression ①
model = LinearRegression() ②
model.fit(x_train, y_train) ③
pred = model.predict(x_val) ④
```

- ① 함수 불러오기: 회귀 모델의 가장 기본이 되는 선형회귀 함수를 불러옵니다.
- ② 모델 선언: LinearRegression 함수로 선형회귀 모델을 선언하고 변수 model에 저장합니다. 함수 LinearRegression의 괄호를 비운 채 선언하면 설정은 기본값으로 하겠다는 의미입니다.
- ③ 학습: 선언한 모델(model)은 fit 메서드로 학습시킵니다. 입력값은 학습 데이터의 x(x\_train)와 y(y\_train)입니다.
- ④ 검증-예측: 학습이 끝난 모델을 검증하려면 predict 메서드로 검증 데이터(학습할 때 사용하지 않은 데이터, x\_val)를 사용해 예측합니다.

머신러닝 모델링 절차는 딥러닝과 큰 틀에서 매우 유사합니다. 좀 더 자세한 내용은 3일 차 회귀 모델링에서 설명합니다.



### 선형회귀

선형회귀(Linear Regression)는 머신러닝에서 가장 기본이 되는 알고리즘으로 입력값(x)과 출력값(y) 사이의 선형 관계를 찾아 예측하는 모델입니다. 주어진 데이터를 가장 잘 설명하는 직

선을 그리는 것이 핵심이며, 직선은 수학적으로  $y = wx + b$ 라는 1차 함수로 표현됩니다. 여기서  $w$ 는 기울기(가중치),  $b$ 는 절편입니다. 모델을 학습시키면서 가장 적절한  $w$ 와  $b$ 의 값을 결정합니다.

또한 선형회귀는 딥러닝의 기본 개념을 이해하기 위한 중요한 출발점입니다. 딥러닝 모델은 여러 개의 입력과 층을 쌓아 복잡한 비선형 문제를 해결하지만, 마지막 출력층에서는 선형회귀와 같은 구조를 사용하는 경우가 많습니다. 또한 선형회귀처럼 오차를 계산한 다음, 오차를 줄이기 위해 가중치를 조정하는 과정(최적화)은 딥러닝도 동일합니다. 따라서 선형회귀를 제대로 이해하면 딥러닝의 학습 구조도 훨씬 쉽게 받아들일 수 있습니다.

그런데 한 가지 주의해서 살펴볼 지점이 있습니다. 모델을 학습할 때는 `x_train`과 `y_train`을 이용하고, 예측할 때는 왜 `x_val`만 이용할까요? 우리는 공부할 때 답을 맞춰 가면서 문제를 풀지요? 모델도 마찬가지입니다. 학습할 때는 문제(`x_train`)와 답(`y_train`)이 필요합니다. 그러나 학습이 모두 끝난 다음, 실력을 검증하기 위해 모의고사를 볼 때는 당연한 이야기지만 문제만 주어진 상태에서 풀니다. 모델링도 마찬가지로 검증할 때는 시험 문제(`x_val`)만 제공하는 거죠. 그렇다면 `y_val`은 언제 사용할까요? 이것은 바로 이어지는 모델 평가 지표에서 설명합니다.

## 회귀 모델 평가

`x_val`이 모의고사 문제라면 `y_val`은 이 시험의 목푫값(target)입니다. 시험 문제를 풀면 채점해서 점수를 매기는 과정이 필요하듯이 모델 역시 학습을 평가합니다. 이를 '모델 평가'라고 합니다.

### 모델 평가하기

회귀 모델을 평가하는 대표적인 4가지 지표 코드를 작성하고 실행합니다.

#### CODE

```
print('RMSE:', root_mean_squared_error(y_val, pred)) ①
print('R2:', r2_score(y_val, pred))
print('MAE:', mean_absolute_error(y_val, pred))
print('MAPE:', mean_absolute_percentage_error(y_val, pred))
```

#### OUTPUT

```
RMSE: 2.1824586788058977
R2: 0.8568542870042719
MAE: 1.5030135035019552
MAPE: 0.23452930250376364
```

① `root_mean_squared_error(y_val, pred)`: 목푫값(`y_val`)과 예측값(`pred`)을 순서대로

입력해 성능 평가 지표 RMSE를 계산합니다. R2, MAE, MAPE의 값도 각각 다른 함수를 이용해 같은 방식으로 얻습니다.

sklearn에서 제공하는 성능 평가 함수들은 목표값과 예측값순으로 입력합니다. 특히 MAPE는 입력 순서에 따라 평가 결과가 달라지므로 순서에 유의해야 합니다.

### 회귀 모델 평가 1: 오차의 크기와 오차율

실제 지표를 이해하기 전에 몇 가지 개념을 이해하는 게 필요합니다. 우선 오차를 표현할 때 자주 사용하는 기호들을 살펴보겠습니다.

- $y$ : 목표값(Ground Truth, 정답 데이터)
- $\hat{y}$ : 예측값(Predicted Value, y-hat이라고 읽음)
- $\bar{y}$ : 평균값(Mean Value, y-bar라고 읽음)

이 기호들은 통계나 딥러닝 모델 평가에서 자주 쓰이는 기본 표현들입니다.

모델을 평가할 때 사용하는 기본 방법 중의 하나는 오차를 측정하는 일입니다. 오차는 목표값과 예측값의 차이입니다.

$$\text{error} = y - \hat{y}$$

오차가 양수이면 모델이 예측을 실제보다 작게, 음수이면 실제보다 크게 예측한 경우입니다.

그리고 오차의 크기를 계산하는 방법은 두 가지입니다.

- 제곱 오차(Squared Error):  $(y - \hat{y})^2$
- 절댓값 오차(Absolute Error):  $|y - \hat{y}|$

두 방법 모두 오차의 크기를 나타냅니다. 특히 제곱 오차는 큰 손실값에 더 큰 패널티를 주어 계산합니다.

그리고 오차율은 다음과 같이 표현합니다.

- 상대 절대 오차(Relative Absolute Error):  $\left| \frac{y - \hat{y}}{y} \right|$

상대 절대 오차는 목표값에 대한 상대적 비율입니다.

[표 1-3]은 지금까지 설명한 오차의 크기 두 가지와 오차율을 계산하는 예제입니다.

[표 1-3] 오차 계산 예제

| 예측값( $\hat{y}$ ) | 목표값( $y$ ) | 오차 | 제곱 오차 | 절댓값 오차 | 상대 절대 오차 |
|------------------|------------|----|-------|--------|----------|
| 1                | 1          | 0  | 0     | 0      | 0        |
| 2                | 1          | -1 | 1     | 1      | 1        |
| 0                | -1         | -1 | 1     | 1      | 1        |
| 3                | 4          | 1  | 1     | 1      | 1/4      |
| 2                | 5          | 3  | 9     | 3      | 3/5      |

앞에서 회귀 모델을 평가할 때 사용한 지표들은 이들 제곱 오차, 절댓값 오차, 상대 절대 오차를 이용한 것입니다.

- MSE(Mean Squared Error):  $\frac{\sum (y - \hat{y})^2}{n}$ . 제곱 오차의 평균값입니다.
- RMSE(Root Mean Squared Error):  $\sqrt{\frac{\sum (y - \hat{y})^2}{n}}$ . MSE는 오차를 제곱해서 계산한 값이므로 원래 값과 단위가 동일하도록 MSE에 제곱근을 취해 구합니다.
- MAE(Mean Absolute Error):  $\frac{\sum |y - \hat{y}|}{n}$ . 절댓값 오차(Absolute Error)의 평균을 계산한 값입니다.
- MAPE(Mean Absolute Percentage Error):  $\frac{\sum \left| \frac{y - \hat{y}}{y} \right|}{n}$ . 상대 절대 오차의 평균입니다.



#### 평균을 나타내는 수식 $\frac{\sum \square}{n}$

수식이 나오면 부담스러워 하는 사람이 많습니다. 하지만 수학 기호를 조금만 이해하면 상당히 많은 수식이 눈에 들어옵니다. 앞에서 설명한 평가 지표의 수식을 보면 공통적으로 보이는 수식 형태가 있습니다. 바로  $\frac{\sum \square}{n}$ 이지요. 수식에서  $\frac{\sum \square}{n}$  같은 형태가 있으면 평균으로 읽으면 됩니다. 산술 평균은 모두 더해서 개수로 나누어 계산합니다. 모두 더하라는 수학 기호가  $\sum \square$ 이고,  $n$ 은 개수를 의미하므로  $\frac{\sum \square}{n}$ 과 같은 식은 “평균을 구한다는 의미구나”라고 생각하면 됩니다.

앞서 실행했던 회귀 모델의 성능 평가 결과를 표로 정리하고 그 의미를 살펴보겠습니다(소수점 넷째 자리에서 반올림).

[표 1-4] 성능 평가 지표의 값과 의미

| 지표   | 값     | 의미                                                                   |
|------|-------|----------------------------------------------------------------------|
| RMSE | 2.182 | 오차의 평균적인 양을 가능해 볼 수 있습니다. 그러나 오차의 제곱으로 계산된 값이므로 오차의 양과 딱 일치하지는 않습니다. |
| MAE  | 1.503 | ‘평균 오차’라고 합니다. 회귀 모델로 예측할 때 발생하는 평균 오차가 1.5(1,500달러) 정도 된다는 의미입니다.   |
| MAPE | 0.235 | ‘평균 오차율’이라고 합니다. 어떤 모델은 매출액을 예측할 때 평균적으로 23.5% 정도 오차가 발생한다는 의미입니다.   |

오차의 양과 오차율의 평가 지표를 살펴보았습니다. 이어서 회귀 모델의 설명력이 라고 하는 평가 지표인  $R^2$  Score를 설명합니다.

#### 회귀 모델 평가 2: 모델의 설명력( $R^2$ Score)

모델을 평가할 때 중요한 것이 기준점입니다. 기준 대비 얼마나 성능이 좋은지 비교 평가를 합니다. 기존 모델이 있다면 현재 모델이 기존 모델보다 얼마나 성능이 개선되었는지 비교하고 평가할 수 있습니다. 또한 기존 모델의 존재 여부와 상관없이 최소한의 기준을 평균으로 삼을 수도 있습니다.

예를 들어 판매량을 예측하는 모델을 만든다고 합시다. 이때 학습 데이터의 평균 판매량이 10개라면 항상 10으로만 예측하는, 즉 평균값으로 예측하는 모델을 기준 점으로 삼습니다. 그리고 회귀 모델을 만들 때 최소한 평균값으로 예측하는 모델보다는 성능이 좋기를 기대합니다.

$R^2$  Score는 회귀 모델이 단순히 평균값으로 예측하는 모델보다 얼마나 더 성능이 개선되었는지를 평가하는 지표입니다. 이를 계산하는 수식은 조금 복잡하므로 여기서는 개념만 단순화해 설명합니다.

- 평균값 예측 모델에서 발생하는 오차를 전체 오차(Total Error)라고 합니다.
- 회귀 모델에서 발생하는 오차를 회귀 오차(Regression Error)라고 합니다.

$$R^2 \text{ Score} = (\text{전체 오차} - \text{회귀 모델 오차}) / \text{전체 오차}$$

다시 말하면  $R^2$  Score는 평균으로 예측한 값보다 회귀 모델이 얼마나 더 잘 예측하는지 나타내는 평가 지표입니다. 특별히  $R^2$  Score를 모델의 설명력 혹은 결정계수라고 합니다.

앞에서 만든 회귀 모델의  $R^2$  Score 값은 0.857(0.856...에서 반올림)이었습니다. 평균으로 예측했을 때의 오차를 기준으로 할 경우, 이 회귀 모델은 “약 85.7%를 줄여 주었다” 또는 “85.7% 더 잘 설명한다”라는 의미입니다. 비즈니스 관점으로 이해하기가 쉽지는 않지요? 일단 이 정도로 평가 지표는 어떤 것들이 있고 그 의미는 무엇인지 대략 살펴보았습니다.

지금까지 머신러닝의 회귀 모델링 과정을 처음부터 끝까지 수행했습니다. 간단한 코드들로 구성했지만, 일반적인 모델링 흐름을 이해하는 데는 부족함이 없습니다. 다시 한번 전체적인 흐름을 요약하면 다음과 같습니다.

- 환경 준비: 필요한 라이브러리와 함수들을 불러왔습니다.
- 데이터 준비: ‘csv’ 파일로 저장된 데이터를 불러와 데이터프레임으로 저장하고 x와 y 분할, 학습 데이터와 검증 데이터로 분할, 스케일링을 수행했습니다.
- 모델링 및 평가: 선형회귀 모델을 선언한 다음, 학습(fit)하고 예측(predict)하고 평가했습니다.

## 분류 모델링 예제: 대학원 합격 여부 예측

합격 이진분류  
불합격

회귀 모델에 이어 대학원 합격 여부를 예측하는 분류 모델을 코드로 작성합니다. 앞서 만든 구글 드라이브 폴더에서 마우스 오른쪽 버튼을 클릭한 다음, [더 보기] - [Google Colaboratory] 메뉴를 차례로 선택해 새 코랩 파일을 생성합니다. 파일 이름은 ‘1일차\_머신러닝실습2.ipynb’로 변경합니다.

이번 예제의 목표는 학생 성적, 시험 점수, 출신 학교 수준 등의 데이터로 그 학생이 대학원에 합격할 가능성이 있는지 예측하는 일입니다. 말 그대로 합격(True)이나 불합격(False)이냐를 판별하는 이진 분류(Binary Classification) 문제입니다.

### 환경 준비

불러올 라이브러리는 ‘1일차\_머신러닝실습1.ipynb’의 환경 준비와 동일합니다.

다음 코드를 작성하고 실행합니다.

#### CODE

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.metrics import *
from sklearn.preprocessing import StandardScaler, MinMaxScaler
```

pandas, numpy, matplotlib, seaborn은 기본적으로 라이브러리 전체를 불러옵니다. sklearn 라이브러리에서는 필요한 함수만 선별해 불러옵니다. 그러나 반드시 이렇게 불러와야 하는 것은 아니며 개발자의 코딩 스타일에 맞게 얼마든지 다르게 작성할 수 있습니다.

## 데이터 전처리

데이터를 모델링에 적합한 형태로 정리하는 전처리 단계입니다. 데이터를 불러와 다음과 같은 순서로 전처리를 수행합니다.

- ① 데이터 준비 **읽기**
- ② x, y 분할 **데이터 / 레이블 분리**
- ③ **가변수화** **원핫**
- ④ train, validation 분할 **test**
- ⑤ 스케일링

‘1일차\_머신러닝실습1’의 전처리와 달리 ‘가변수화’ 단계가 추가되었습니다.

### 데이터 준비

데이터 파일의 경로(path)를 지정한 다음, 판다스의 read\_csv 메서드로 데이터를 불러와 데이터프레임으로 저장합니다(data2). 계속해서 head 메서드로 상위 5건의 데이터를 조회합니다.

```
CODE
path = "https://bit.ly/graduate_csv"
data2 = pd.read_csv(path)
data2.head()
```

이 데이터 세트는 대학원 진학을 시도한 학생들의 정보를 담고 있습니다. 조회 결과(표 1-5))를 보면 각각의 행은 한 명의 지원자이며 입력변수는 모두 4개입니다.

[표 1-5] 대학원 진학 시험 응시자의 데이터  
label data

| admit | gre | gpa  | rank |
|-------|-----|------|------|
| 0     | 380 | 3.61 | 3    |
| 1     | 660 | 3.67 | 3    |
| 1     | 800 | 4.00 | 1    |
| 1     | 640 | 3.19 | 4    |
| 0     | 520 | 2.93 | 4    |

rank <- 범주형

- admit: 합격 여부(1: 합격, 0: 불합격)→목표값
- gre: GRE 시험 점수(대학원 입학 시험)
- gpa: 학부 평균 성적(Grade Point Average)
- rank: 지원자의 출신 학교 랭킹(1~4 범주)

이 데이터의 admit이 바로 예측하려는 목표값이고 나머지 변수들은 예측에 사용할 입력값입니다.

## x, y 나누기

데이터를 x와 y로 나눕니다. 이 구조로 데이터를 분리하면 이후 전처리와 모델 학습이 훨씬 깔끔하게 이루어집니다.

### CODE

```
target = 'admit'
x = data2.drop(target, axis=1)
y = data2.loc[:, target]
```

코드는 이미 앞에서 설명했습니다. 33쪽을 참조하기 바랍니다.

## 가변수화

이번 예제의 데이터 전처리 단계에서 주의할 변수는 범주형인 rank입니다. 범주형 변수 rank는 모델링하기 전에 숫자로 변환하는 전처리 작업이 필요합니다. 이를 가변수화라고 합니다.

모델은 데이터에서 패턴을 찾으면서 수학 연산을 하는데, 범주형 데이터로는 수학 연산을 직접 할 수 없기 때문에 가변수화를 합니다. 예를 들어 성별 데이터의 경

우 남자나 여자를 직접 수학적식에 입력해 계산할 수는 없습니다. 남자에 3을 곱하거나 여자에 5를 더할 수는 없기 때문입니다. 그래서 모델링을 하기 전에 범주형 데이터는 반드시 숫자로 변환합니다.

여기서 한 가지 의문을 가질 분도 있을 것 같습니다. 이미 “변수 rank의 값은 숫자 1, 2, 3, 4로 되어 있는데, 왜 가변수화를 하지?”라고 말합니다. 그런데 rank에 저장된 숫자는 데이터 타입이 숫자일 뿐, 의미는 숫자가 아닌 범주입니다. 즉, rank가 2라고 해서 rank가 1인 값보다 2배 크다는 의미는 아닙니다. 그래서 범주형 변수가 숫자로 표현되어 있더라도 이 숫자를 바로 계산에서 사용할 수 없습니다.

빈 셀에서 다음 코드를 입력합니다.

#### CODE

```
cat_cols = ['rank'] ①
x = pd.get_dummies(x, columns = cat_cols, drop_first = True) ②
x.head()
```

원-핫 인코딩

#### OUTPUT

|   | gre | gpa  | rank_2 | rank_3 | rank_4 |
|---|-----|------|--------|--------|--------|
| 0 | 380 | 3.61 | False  | True   | False  |
| 1 | 660 | 3.67 | False  | True   | False  |
| 2 | 800 | 4.00 | False  | False  | False  |
| 3 | 640 | 3.19 | False  | False  | True   |
| 4 | 520 | 2.93 | False  | False  | True   |

① 범주형 변수를 리스트 형식으로 저장합니다.

② get\_dummies 메서드를 이용해 범주형 변수를 가변수(0 혹은 1) 형태로 변환합니다.

- x: 가변수화할 대상 데이터 세트
- columns=cat\_cols: 범주형 변수 목록. ①에서 리스트로 저장한 범주형 변수.
- drop\_first=True: 가변수화한 다음 첫 번째 범주를 제거해 다중 공선성(Multicollinearity) 문제를 방지(예: rank\_1, rank\_2, rank\_3, rank\_4 중 rank\_1을 제거하고 나머지 3개만 사용)

코드에서 가변수화를 수행하고 head 메서드로 상위 5건의 결과를 출력했습니다. rank 변수는 rank\_2, rank\_3, rank\_4 세 개의 열로 변환되어 있습니다. 코드에서 설명했듯이 drop\_first 옵션을 True로 설정하면 rank\_1은 결과에서 제외됩니다.

가변수화된 값은 True/False로 표시되어 있는데, 모델을 학습할 때는 True는 1, False는 0으로 바뀌어 처리됩니다.



### 다중 공선성(Multicollinearity)이란?

다중 공선성 문제는 여러 개의 입력변수(x) 사이에 강한 상관관계가 있을 때 발생하는 문제입니다. 주로 선형회귀나 로지스틱 회귀 모델에서 발생합니다. 입력변수 사이에 강한 상관관계가 있는 경우 모델링 과정에서 각각의 입력변수와 목표값(y)의 개별적인 관계를 제대로 학습하지 못해 예측 성능이 떨어질 수 있습니다.

가변수화할 때 생성된 변수 중 하나를 제거하는 이유는 이러한 다중 공선성 문제를 피하기 위해서입니다. 예컨대 성별(남, 여) 범주형 변수를 가변수화하면 [표 1-6]과 같은 결과를 얻습니다.

[표 1-6] 성별 변수의 가변수화 예시

| 남자    | 여자    |
|-------|-------|
| True  | False |
| False | True  |
| True  | False |

결국값을 보면 남자의 값이 True면 여자는 False, 반대로 남자가 False면 여자는 무조건 True 일 수밖에 없습니다. 즉, 남자 또는 여자의 열 중 하나를 제외해도 성별 변수의 값을 표현할 수 있습니다. 둘 다 있을 필요가 없으며 오히려 있다면 정보의 중복이 발생합니다.

다시 대학원 합격 여부 예측 데이터로 넘어가서 rank 변수는 4개의 범주로 구성되어 있습니다. 이 변수를 가변수화할 때 범주 3개의 값을 알면 나머지 범주값은 자동으로 결정됩니다. 만일 4개의 범주로 구성하면 정보의 중복이 발생합니다. 정보 중복은 다중 공선성 문제를 일으킵니다. 따라서 정보 중복을 방지하기 위해 가변수화 함수인 `get_dummies`에서 `drop_first` 옵션을 True로 설정합니다.

그러나 선형회귀, 로지스틱 회귀 모델 외에는 다중 공선성이 큰 문제가 되지 않습니다. 특히 딥러닝에서는 다중 공선성이 고려 대상이 아니기 때문에 크게 신경 쓸 필요가 없습니다.

## 데이터 분할 및 스케일링

이어서 데이터를 학습 데이터와 검증 데이터로 분할하고 스케일링을 수행합니다

### CODE

```
x_train, x_val, y_train, y_val = train_test_split(x, y, test_size = .3,
                                                random_state=20)
scaler = MinMaxScaler()
```

```
x_train = scaler.fit_transform(x_train)
x_val = scaler.transform(x_val)
```

코드의 설명은 33~34쪽을 참조 바랍니다. 이제 모델링을 수행하기 위한 데이터의 준비를 완료했습니다.

## 모델링: 학습, 예측

이번에는 로지스틱 회귀(Logistic Regression) 모델을 사용합니다. 로지스틱 회귀는 가장 기본적인 분류 알고리즘입니다. 특히 딥러닝으로 이진 분류 모델을 만들 때는 로지스틱 회귀의 원리를 바탕으로 하므로 반드시 기억해야 할 중요한 알고리즘입니다. 로지스틱 회귀는 4일 차에서 좀 더 자세히 다룹니다.

### CODE

```
from sklearn.linear_model import LogisticRegression ①
model = LogisticRegression()
model.fit(x_train, y_train)
pred = model.predict(x_val)
```

① LogisticRegression 함수를 불러와 모델을 선언합니다. 분류 모델의 가장 기본적인 알고리즘인 로지스틱 회귀 모델로 선언합니다. 나머지 코드의 설명은 35쪽과 동일합니다.

로지스틱 회귀 모델을 선언하고 학습시키고 예측했습니다.

로지스틱 회귀는 이름에 ‘회귀’가 들어 있지만, 실제로는 분류(Classification) 문제를 해결하는 데 사용하는 머신러닝 알고리즘입니다. 입력값(x)과 목표값(y) 사이의 관계를 학습한다는 점에서는 선형회귀와 비슷하지만, y가 연속적인 수치가 아니라 특정 범주(예: 합격/불합격, 스팸/정상)라는 점에서 다릅니다. 로지스틱 회귀는  $y = wx + b$ 의 결과를 시그모이드(Sigmoid) 함수에 통과시켜 0과 1 사이의 확률 값으로 바꾸고 이 확률을 기준으로 범주를 분류하는 모델입니다. 로지스틱 함수라고도 불리는 시그모이드는 굉장히 중요한 함수로 4일 차에서 좀 더 자세히 설명합니다.

## 분류 모델 평가 방법

이번에는 분류 모델을 어떻게 평가하는지 살펴보겠습니다. 방금 생성한 분류 모델의 평가 코드를 작성하고 평가 방법에는 어떤 것이 있는지, 결과는 어떻게 해석하는지 설명합니다.

## 분류 모델 평가하기

분류 모델은 혼동 행렬(Confusion Matrix)과 sklearn에서 제공하는 분류 결과 보고서(classification\_report) 함수를 이용해 평가합니다.

다음과 같이 코드를 작성하고 실행 결과를 살펴보겠습니다.

### CODE

```
print(confusion_matrix(y_val, pred)) ①
```

### OUTPUT

```
[[74 10]
 [26 10]]
```

① 혼동 행렬(confusion\_matrix) 함수의 입력으로 검증 데이터의 목표값(y\_val)과 예측값(pred)을 순서대로 받습니다.

코드의 실행 결과로 나온 혼동 행렬을 이용해 모델의 성능을 다양하게 평가할 수 있습니다.

여러 평가 결과를 일목요연하게 정리해서 보여 주는 분류 보고서 함수를 출력합니다.

### CODE

```
print(classification_report(y_val, pred)) ①
```

### OUTPUT

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.74      | 0.88   | 0.80     | 84      |
| 1            | 0.50      | 0.28   | 0.36     | 36      |
| accuracy     |           |        | 0.70     | 120     |
| macro avg    | 0.62      | 0.58   | 0.58     | 120     |
| weighted avg | 0.67      | 0.70   | 0.67     | 120     |

① 분류 결과 보고서(classification\_report) 함수는 혼동 행렬을 이용한 다양한 분류 모델 평가 지표를 간단한 보고서 형태로 출력합니다. 이 함수도 목표값과 예측값순으로 입력해야 합니다.

참고로 전처리 단계에서 사용한 train\_test\_split 함수는 데이터를 랜덤하게 분할합니다. 하지만 random\_state=20으로 옵션을 지정했으므로 독자도 필자와 동일한 결과가 나왔을 겁니다. 하지만 실제 모델링에서는 random\_state 같은 옵션을 사용하지 않습니다. 왜냐하면 실전(운영 환경)에서는 다양한 데이터가 생성될 수 있기에 동일한 조건으로 모델링을 수행할지라도 성능은 조금씩 달라질 수 있습니다. 이

는 자연스러운 현상으로 성능 평가 결과의 변동성(Variance)으로 이해하고 받아들여야 합니다. 공부하는 학생이 모의고사를 여러 회 풀면 풀 때마다 조금씩 점수가 달라집니다. 시험 문제가 다르니 성적에도 편차가 있기 마련입니다.

이제 분류 모델에 대한 성능 평가 결과를 좀 더 자세히 살펴보겠습니다.

#### 분류 모델 평가 1: 혼동 행렬

분류 모델을 평가하려면 먼저 혼동 행렬(Confusion Matrix)을 만들고 혼동 행렬에서 다양한 평가 지표를 계산해야 합니다. 먼저 혼동 행렬을 만드는 과정을 설명합니다.

쉽게 이해할 수 있도록 검증 데이터 8건의 예측 결과를 [표 1-7]에 담았습니다.

[표 1-7] 모델의 예측 결과 예시

| 예측값 | 목표값 |
|-----|-----|
| 합격  | 합격  |
| 불합격 | 불합격 |
| 합격  | 불합격 |
| 불합격 | 합격  |
| 불합격 | 불합격 |
| 불합격 | 불합격 |
| 합격  | 합격  |
| 불합격 | 불합격 |

[표 1-7]의 결과는 다음과 같이 4가지 경우로 집계할 수 있습니다.

- 실제로 합격인데, 예측도 합격인 경우: 2건
- 실제로 합격인데, 예측은 불합격인 경우: 1건
- 실제로 불합격인데, 예측은 합격인 경우: 1건
- 실제로 불합격인데, 예측도 불합격인 경우: 4건

집계한 숫자를 [표 1-8]로 다시 정리한 것이 교차 집계표, 즉 혼동 행렬입니다.

[표 1-8] 간단한 교차 집계표 예시

|     |     | 예측값 |    |   |
|-----|-----|-----|----|---|
|     |     | 불합격 | 합격 | 계 |
| 목표값 | 불합격 | 4   | 1  | 5 |
|     | 합격  | 1   | 2  | 3 |
|     | 계   | 5   | 3  | 8 |

이 혼동 행렬의 각 숫자를 이용해 모델을 평가하는데, 중요한 평가 지표들을 살펴보면 다음과 같습니다.

- 정분류율(Accuracy): 혼동 행렬의 전체 데이터(8명) 중에서 합격(2명)과 불합격(4명)을 모두 맞춘 비율입니다( $\frac{4+2}{8} = 0.75$ ).
- 재현율, 정밀도, F1 Score는 불합격과 합격 관점에서 각각 계산해야 합니다.
- 재현율(Recall): 재현율을 합격 관점에서 설명하면 실제 합격한 사람(3명) 중에서 합격을 예측해서 맞춘 사람 수(2명)의 비율입니다( $\frac{2}{3} = 0.67$ ). 재현율은 목표값을 기준으로 얼마나 맞췄는지를 평가한다는 점이 특징입니다.
- 정밀도(Precision): 역시 합격 관점에서 설명하면 합격으로 예측한 사람(3명) 중에서 실제로 맞춘 사람 수(2명)의 비율입니다( $\frac{2}{3} = 0.67$ ). 정밀도는 예측값을 기준으로 얼마나 맞췄는지를 평가한다는 점이 특징입니다.
- F1 Score: 재현율과 정밀도의 조화평균(Harmonic Mean)입니다. 이 지표는 재현율과 정밀도 두 지표를 함께 고려해 평가합니다( $\frac{2 \times \text{Recall} \times \text{Precision}}{\text{recall} + \text{precision}} = \frac{2 \times 0.67 \times 0.67}{0.67 + 0.67} = 0.67$ ).

분류 모델을 평가하는 4가지 성능 지표는 모두 “모델이 얼마나 잘 맞췄는가?”와 관련되어 있기에 1에 가까울수록 성능이 좋은 모델입니다. 정분류율은 모델의 정확도이므로 쉽게 이해가 되는데, 재현율과 정밀도는 여전히 혼란스럽나요? 일상과 관련된 예시로 다시 한번 설명하겠습니다.

우선 재현율은 목표값을 기준으로 얼마나 맞췄는지, 정밀도는 예측값을 기준으로 얼마나 맞췄는지를 평가하는 지표라는 점을 기억하며 다음 내용을 천천히 읽기 바랍니다.

코로나 진단 키트로 100명을 검사하는데, 이 가운데 실제 코로나 환자는 15명이

라고 가정하겠습니다. 진단 키트 검사 결과, 코로나 양성인 20명이 나왔습니다. 그런데 코로나 양성 20명 중에서 진짜 코로나 확진자는 7명이고 정상은 13명입니다. 이를 혼동 행렬로 정리하면 [표 1-9]와 같습니다.

[표 1-9] 코로나 진단 검사 혼동 행렬 예시

|     |    | 예측값 |    |     |
|-----|----|-----|----|-----|
|     |    | 음성  | 양성 | 계   |
| 목표값 | 정상 | 72  | 13 | 85  |
|     | 확진 | 8   | 7  | 15  |
|     | 계  | 80  | 20 | 100 |

확진자 혹은 양성 관점에서 이 진단 키트의 재현율과 정밀도를 계산합니다.

- 재현율: “이 진단 키트는 코로나 확진자를 얼마나 잘 찾아?”라는 질문에 대한 답이 바로 재현율입니다. 여기서는 실제 확진자 15명 중에서 7명을 찾아냈습니다.
- 정밀도: “이 진단 키트는 양성 예측자를 얼마나 잘 맞춰?”라는 질문은 정밀도가 얼마인지를 묻는 것입니다. 여기서는 양성으로 예측한 20명 중에서 7명을 맞췄습니다.

이제 조금 이해가 되나요? 이처럼 재현율과 정밀도는 비슷한 듯하지만 관점이 다른 성능 평가 지표이기에 현업에서도 처음에는 많이 혼동하는 편입니다.

실무에서 이 두 값은 보완 관계에 있기 때문에 함께 살펴봐야 합니다. 필자가 어떤 코로나 확진자도 모두 찾을 수 있는 진단 키트를 만든다고 한다면 독자는 아마도 필자를 이상하게 생각할 겁니다. 이 말은 재현율이 1인 진단 키트를 만들 수 있다는 의미입니다. 곰곰이 생각해 보면 이런 진단 키트를 만드는 일은 대단히 쉽습니다. 검사한 모든 사람을 그냥 ‘양성’으로 분류하면 1이 됩니다. 확진자를 한 명도 놓치지 않았으니까요. 어처구니없는 일이죠? 그런 검사 키트를 사용할 의사는 아무도 없을 겁니다. 이처럼 양성으로 극단적으로 분류하면 양성이라고 예측한 시도가운데 실제 양성을 맞춘 비율, 즉 정밀도가 형편없이 낮아집니다. 그래서 이 두 지표는 균형 있게 살펴야 합니다.

그리고 두 지표를 하나로 요약한(조화평균으로 계산한) 지표가 바로 F1 Score입니다.

## 분류 모델 평가 2: 분류 결과 보고서 함수

분류 결과 보고서 함수(classification\_report)는 혼동 행렬에서 계산한 다양한 성능 지표를 요약 보고서 형태로 제공합니다.

앞에서 대학원 합격 예측 예제의 성능 평가 결과를 다시 가져오겠습니다. 혼동 행렬 코드를 실행하면 나오는 결과를 표 형태로 정리하면 [표 1-10]과 같습니다.

[표 1-10] 대학원 합격 여부 예측 모델의 혼동 행렬

|     |     | 예측값 |    |     |
|-----|-----|-----|----|-----|
|     |     | 불합격 | 합격 | 계   |
| 목표값 | 불합격 | 77  | 3  | 80  |
|     | 합격  | 34  | 6  | 40  |
|     | 계   | 111 | 9  | 120 |

검증 데이터 120건을 모델로 예측하고 집계했습니다. 혼동 행렬의 각 요소에 해당하는 4개의 숫자(77, 3, 34, 6)로 다양한 성능 지표를 계산할 수 있다고 설명했습니다. 이를 classification\_report 함수를 이용해 한꺼번에 보여 주는 통계가 분류 결과 보고서입니다.

[그림 1-15]와 같이 함수 classification\_report의 결과를 보면 0은 불합격, 1은 합격을 의미합니다. 여러 결과값 중에서 박스로 표시한 숫자를 직접 계산해 보겠습니다.

|                    |      |           |        |          |         |
|--------------------|------|-----------|--------|----------|---------|
| [[77 3]<br>[34 6]] |      |           |        |          |         |
|                    |      | precision | recall | f1-score | support |
| 0                  | 0.69 | 0.96      | 0.81   |          | 80      |
| 1                  | 0.67 | 0.15      | 0.24   |          | 40      |
| accuracy           |      |           | 0.69   |          | 120     |
| macro avg          | 0.68 | 0.56      | 0.53   |          | 120     |
| weighted avg       | 0.68 | 0.69      | 0.62   |          | 120     |

[그림 1-15] 검증 데이터에 대한 예측 결과

- 정분류율: 전체 정분류율은 0.69입니다.  $\frac{77+6}{77+3+34+6} = 0.69$ .
- 재현율: 불합격자(0) 관점에서 실제 불합격자(80명) 중에서 불합격으로 예측해 맞춘 사람(77명)의 비율이 0.96입니다.  $\frac{77}{77+3} = 0.96$ .

- ③ 정밀도: 합격자 관점에서 합격으로 예측한 사람(9명) 중에서 실제 합격자(6명)의 비율입니다.  $\frac{6}{3+6} = 0.67$
- ④ F1 Score: 합격자 관점에서 재현율(0.15)과 정밀도(0.67)의 조화평균 값인 0.24입니다.  $\frac{2 \times 0.15 \times 0.67}{0.15 + 0.67} = 0.24$

이렇게 해서 분류 모델을 어떻게 평가하는지 살펴보았습니다. 모델을 만든 후 검증 데이터로 평가하는 일은 굉장히 중요합니다. 마치 공부를 열심히 한 후 모의고사를 통해 실력을 확인하듯이 모델의 성능을 객관적으로 확인할 수 있는 방법이기 때문입니다.

이번 차시에서 소개한 여러 지표는 이후 딥러닝 모델을 구현하면서 데이터와 비즈니스 관점에서 어떻게 해석하는지 차근차근 소개합니다.

## 정리

이번 차시에서는 머신러닝의 개념부터 실습 흐름까지 빠르게 훑어보며 딥러닝 학습에 앞서 꼭 필요한 기초를 다졌습니다. 인공지능(AI), 머신러닝(ML), 딥러닝(DL)의 개념적 차이를 시작으로 머신러닝이 데이터에서 패턴을 어떻게 학습하고 이를 바탕으로 어떻게 예측하고 분류하는지 다양한 예시로 살펴보았습니다. 특히 머신러닝의 대표적인 학습 방식인 지도학습, 비지도학습, 강화학습을 소개하면서 이 책에서는 지도학습을 중심으로 내용을 전개할 것임을 안내했습니다.

또한 머신러닝 모델이란 무엇이며 왜 '표본으로 전체를 추정하는 일'이 중요한지, 모델링 과정이 단순한 계산이 아닌 데이터와 알고리즘, 그리고 비즈니스 문제의 이해를 함께 요구하는 복합적인 작업인지를 설명했습니다. 이를 바탕으로 데이터 분석 과정을 체계적으로 수행할 수 있도록 돕는 표준 방법론인 CRISP-DM의 여섯 단계를 설명했습니다. 이 방법론은 이후 독자가 실제 프로젝트를 진행할 때 든든한 나침반이 되어줄 것입니다.

실습에서는 구글 코랩 환경에서 머신러닝 모델링의 전체적인 흐름을 경험했습니다. 데이터 전처리의 핵심 요건(결측값 처리, 가변수화, 스케일링), 학습용 데이터와 검증용 데이터의 분할, 모델 설계, 학습, 예측, 평가까지 전 과정을 실습으로 익혔습니다. 또한 회귀(숫자 예측)와 분류(범주 예측) 모델에 각각 어떤 평가 지표(RMSE,  $R^2$ , MAE, Accuracy, Precision, Recall, F1 Score 등)를 사용하는지도 구체적

으로 다뤘습니다.

이제 독자 여러분은 머신러닝이라는 큰 흐름을 정리했습니다. 다음 차시부터는  
인공 신경망(Artificial Neural Network)을 중심으로 딥러닝 모델의 핵심 개념과 구  
조, 학습 원리를 하나씩 깊이 있게 탐구하겠습니다.